

University of Padova

Department of Mathematics

Master Thesis in Mathematics for Data Science

Subspace Identification Methods for Extracting Behaviorally Relevant Neural Dynamics

Supervisors

Professor Michele Allegra
University of Padova

Professor Marco Formentin
University of Padova

Master Candidate

Eleonora Mesaglio

Academic Year

2025-2026

To Alberto, Chiara, Eva and Greta,
for their support from afar,
and to Marlon, for his from near.

Abstract

Neural population recordings provide a unique window into how the brain generates behavior, yet transforming these high-dimensional signals into interpretable dynamical models remains a major challenge. A central difficulty is that neural activity reflects a mixture of intrinsic dynamics, behavioral components, and externally driven responses, and standard modeling approaches do not reliably separate these components. Non-preferential modeling extracts dominant patterns from neural activity without regard for behavioral relevance, and therefore often captures structure that is unrelated to the behavioral output of interest. Preferential modeling addresses this limitation by prioritizing the identification of latent states that are directly relevant to behavior, yielding more accurate and interpretable models of neural population dynamics. Methods that account for behavior but not external inputs, however, confound input-driven structure with intrinsic behavior-related dynamics, leading to systematically imprecise estimates. This thesis provides a rigorous and self-contained analysis of two preferential subspace identification algorithms for linear neural population dynamics: Preferential Subspace Identification (PSID) [1] and its input-driven extension, Input Preferential Subspace Identification (IPSID) [2]. Both algorithms are derived within the linear state-space identification framework of Van Overschee and De Moor [3], with the projection geometry examined in detail at each stage. Non-preferential baselines, Non-preferential neural Dynamic Modeling (NDM) [4, 5] and its input-driven counterpart INDM [3, 6], are introduced to contextualize the advantage of preferential identification. The theoretical exposition is complemented by numerical simulations on randomly generated state-space models, which validate the theoretical guarantees of each method. PSID reliably recovers behaviorally relevant dynamics across a diverse set of models, while IPSID achieves more precise eigenvalue recovery and higher behavioral decoding accuracy in the presence of external inputs. Direct comparison confirms that ignoring measured inputs systematically confounds intrinsic and input-driven dynamics, establishing the practical necessity of IPSID in input-driven settings.

Contents

Abstract v

List of figures ix

List of tables xi

Listing of acronyms xiii

1 Introduction 1

2 Neural Dynamic Modeling 5
- 2.1 Non-preferential neural Dynamic Modeling . . . 6
- 2.2 Input Non-preferential neural Dynamic Modeling . . . 7
- 2.3 Preferential Subspace Identification . . . 8
- 2.4 Input Preferential Subspace Identification . . . 9

3 Preliminaries 11
- 3.1 Oblique Projections . . . 11
- 3.2 Singular Vector Decomposition . . . 14
  - 3.2.1 Existence of SVD . . . 14
  - 3.2.2 Truncated SVD . . . 16
- 3.3 Block Hankel Matrices . . . 17
- 3.4 Kalman Filters . . . 19
  - 3.4.1 Time-Varying Kalman Filter . . . 20
  - 3.4.2 Steady-State Kalman Filter . . . 21

4 Preferential Subspace Identification 25
- 4.1 System-Theoretic Matrices . . . 27
- 4.2 Algorithm Outline . . . 29
- 4.3 Algorithm Details . . . 33
  - 4.3.1 Forward Innovation Formulation . . . 34
  - 4.3.2 Least Squares Parameter Estimate . . . 36
  - 4.3.3 Stage 1: Extracting Behaviorally Relevant States . . . 38
  - 4.3.4 Stage 2: Extracting Behaviorally Irrelevant States . . . 42
  - 4.3.5 Final Stage: Parameter Identification . . . 44

5 Input Preferential Subspace Identification 47
5.1 System-Theoretic Matrices . . . 49
5.2 Algorithm Outline . . . 50
5.3 Algorithm Details . . . 55
5.3.1 Forward Innovation Formulation . . . 56
5.3.2 Least Squares Parameter Estimate . . . 59
5.3.3 Stage 1 and 2: Extracting Latent States . . . 60
5.3.4 Final Stage: Parameter Identification . . . 61

6 Numerical Simulations 65
6.1 PSID . . . 66
6.1.1 Experimental Setup . . . 66
6.1.2 Results . . . 70
6.2 IPSID . . . 72
6.2.1 Experimental Setup . . . 73
6.2.2 Results . . . 74
6.3 Direct Comparison of PSID and IPSID . . . 78

7 Conclusions 81
7.1 Summary of Contributions . . . 81
7.2 Limitations . . . 82
7.3 Future Directions . . . 82

References 85

Acknowledgments 87

Listing of figures

6.1 PSID Eignevalue Recovery 70
6.2 PSID Behavior Decoding 71
6.3 PSID Eigenvalue Convergence 72
6.4 IPSID Eigenvalue Recovery 75
6.5 IPSID Behavior Decoding 76
6.6 IPSID Eigenvalue Convergence 76
6.7 PSID versus IPSID Eigenvalue Recovery 79

Listing of tables

6.1 Summary of PSID and IPSID Simulation Results 77

Listing of acronyms

CVA . . . . . Canonical Variate Analysis

DARE . . . . . Discrete Algebraic Riccati Equation

EEG . . . . . Electroencephalography

fMRI . . . . . Functional Magnetic Resonance Imaging

INDM . . . . . Input Non-preferential neural Dynamic Modeling

IPSID . . . . . Input Preferential Subspace Identification

LFADS . . . . . Latent Factor Analysis via Dynamical Systems

MOESP . . . . . Multivariable Output-Error State sPace

N4SID . . . . . Numerical algorithms for Subspace State Space System Identification

NDM . . . . . Non-preferential neural Dynamic Modeling

PSID . . . . . Preferential Subspace Identification

RNN . . . . . Recurrent Neural Network

SNR . . . . . Signal-to-Noise Ratio

SVD . . . . . Singular Value Decomposition

1 Introduction

Brain activity arises from coordinated firing of neural populations, producing electrical and metabolic signals measurable through techniques such as electroencephalography (EEG) and functional magnetic resonance imaging (fMRI). The recorded neural population activity is not generated by a single source; instead, it reflects a rich combination of interacting dynamical processes that unfold simultaneously.

A first major contributor is intrinsic neural dynamics, the activity generated by recurrent local interactions within the recorded brain region itself. These intrinsic dynamics include both behaviorally relevant components that actively contribute to generating the observed behavior and behaviorally irrelevant components that do not support the execution of the measured behavior. Notably, behaviorally relevant intrinsic dynamics typically account for only a low-dimensional subspace of the recorded neural activity. A second contributor is input-driven dynamics, which originate from structured sensory inputs or activity arriving from upstream brain regions. These inputs can carry their own temporal structure, and this structure can appear in the recorded neural signal even though it does not originate within the region under study.

Because these different sources coexist, recorded neural signals are inherently complex. As a result, distinguishing intrinsic behaviorally relevant dynamics from behavior-irrelevant and input-driven ones remains a central challenge in systems neuroscience. Solving this problem is crucial for accurately interpreting how neural circuits give rise to behavior and for avoiding confounds introduced by sensory or upstream inputs. Achieving a clearer separation of neural

dynamics also supports more precise theories of brain function and benefits practical applications such as brain-machine interfaces, neurostimulation, and the identification of biomarkers in neurological disorders [7, 8].

This thesis is grounded in the framework of linear dynamical systems, which model the joint evolution of neural activity, behavior, and inputs over time as a linear state-space system, building on the subspace identification framework reviewed in Van Overschee and De Moor [3]. Non-preferential approaches such as Non-preferential neural Dynamic Modeling (NDM) [4, 5] and its input-driven extension INDM [3, 6] identify latent dynamics from neural activity without prioritizing behavioral relevance and therefore often capture structure that is unrelated to the behavioral output of interest. In contrast, preferential methods explicitly prioritize the identification of latent dynamics that are predictive of behavior, enabling the separation of behaviorally relevant dynamics from behaviorally irrelevant components. Preferential Subspace Identification (PSID), originally proposed by Sani et al. [1], realizes this approach by explicitly extracting the behaviorally relevant latent subspace from neural population data. Its input-aware extension, Input Preferential Subspace Identification (IPSID) [2], further dissociates intrinsic dynamics from input-driven activity in settings where external inputs are measured.

While this thesis does not introduce new methods, it provides a rigorous and self-contained mathematical treatment of both algorithms. PSID extracts the behaviorally relevant latent subspace through an orthogonal projection of the neural data onto the space of past observations and behavior, while IPSID extends this by applying an oblique projection that additionally removes the contribution of measured external inputs before extracting the intrinsic behaviorally relevant dynamics. The algorithmic steps of both methods are derived from first principles, establishing the theoretical properties that allow their preferential behavior, and NDM and INDM are introduced as non-preferential theoretical baselines to contextualize the advantage of preferential identification.

The theoretical contributions are validated through numerical simulations on randomly generated linear state-space models, in which ground truth parameters are available for direct assessment of identification accuracy. The simulations adopt a more general setting than the original papers, introducing additional coupling between model components to provide a more stringent test of both methods. PSID is shown to reliably recover behaviorally relevant dynamics across a diverse set of models, achieving low normalized eigenvalue errors and accurate behavior decoding. IPSID further demonstrates that explicitly accounting for external inputs yields more precise eigenvalue recovery and higher decoding accuracy in input-driven settings. A

direct comparison confirms that applying PSID (which does not account for inputs) to input-driven data leads to systematic confounding of intrinsic and input-driven dynamics, highlighting the practical necessity of IPSID in such settings. Together, these results provide a principled understanding of how preferential and input-aware identification methods can disentangle the multiple sources underlying neural population activity.

Objectives

The primary objectives of this thesis are the following.

1. To provide a complete mathematical derivation of the PSID and IPSID algorithms from first principles.
2. To introduce NDM and INDM as non-preferential theoretical baselines, establishing the conceptual distinction between preferential and non-preferential neural dynamical modeling.
3. To validate the theoretical properties of PSID and IPSID through numerical simulations on randomly generated linear state-space models, assessing eigenvalue recovery accuracy and behavior decoding performance.
4. To demonstrate, through a direct controlled comparison, that ignoring measured external inputs leads to systematic confounding of intrinsic and input-driven dynamics, and that IPSID correctly dissociates the two.

These objectives guide the development of the thesis, whose structure is outlined below.

Thesis Outline

The remainder of this thesis is organized as follows.

- **Chapter 2, Neural Dynamical Modeling**, introduces the modeling framework, presenting NDM, INDM, PSID, and IPSID at a conceptual level and establishing the landscape of methods considered in this thesis.
- **Chapter 3, Mathematical Preliminaries**, develops the theoretical tools required for the subsequent derivations, including oblique projections, singular value decomposition, block Hankel matrices, and Kalman filtering.

- **Chapter 4, Preferential Subspace Identification**, presents the theoretical foundations of PSID and derives its algorithmic steps from the subspace identification framework.
- **Chapter 5, Input-Preferential Subspace Identification**, develops the theoretical foundations and algorithmic derivation of IPSID, extending the PSID framework to input-driven settings.
- **Chapter 6, Numerical Simulations**, evaluates PSID and IPSID through numerical experiments on randomly generated state-space models, assessing eigenvalue recovery accuracy, behavior decoding performance, and the advantage of IPSID in the presence of external inputs.
- **Chapter 7, Conclusions**, summarizes the main findings of the thesis and outlines directions for future work.

Together, these chapters provide a comprehensive development of preferential subspace identification methods, from theoretical foundations to practical validation.

2

Neural Dynamic Modeling

Linear dynamical systems provide a natural framework for modeling the temporal evolution of neural activity. They model neural population activity by identifying low-dimensional latent dynamics that evolve linearly over time. These models can be formulated to treat all neural dynamics equally, or to preferentially focus on dynamics that are relevant to a measured behavior.

This distinction leads to two broad classes of neural dynamics models. Non-preferential methods, such as Non-preferential neural Dynamic Modeling (NDM) and Input Non-preferential neural Dynamic Modeling (INDM), treat all sources of neural dynamics equally, providing a complete characterization of neural dynamics, but failing to distinguish behaviorally relevant from irrelevant components. Preferential methods, Preferential Subspace Identification (PSID) and Input Preferential Subspace Identification (IPSID), overcome this limitation by explicitly prioritizing the identification of behaviorally relevant dynamics. This preferential learning enables a more targeted characterization of the specific neural mechanisms that drive behavior.

In this chapter, we introduce these four modeling approaches, progressing from non preferential to preferential methods. The detailed mathematical formulations of PSID and IPSID will be presented in subsequent chapters.

2.1 Non-preferential neural Dynamic Modeling

Non-preferential neural dynamical modeling (NDM) [4, 5] provides a framework for capturing the intrinsic temporal structure in neural population activity without distinguishing between behaviorally relevant and behaviorally irrelevant dynamics. Because it does not prioritize the learning of any particular dynamical component, NDM is a non-preferential modeling approach. In its standard formulation, NDM represents neural activity through a latent linear dynamical system:

$$\begin{cases} x_{k+1} = Ax_k + w_k \\ y_k = C_y x_k + v_k \end{cases}, \tag{2.1}$$

where $y_k \in \mathbb{R}^{n_y}$ denotes the observed neural activity, $x_k \in \mathbb{R}^{n_x}$ is a low-dimensional latent state, and $w_k \in \mathbb{R}^{n_x}$ and $v_k \in \mathbb{R}^{n_y}$ represent process noise and measurement noise, respectively. The state transition matrix $A \in \mathbb{R}^{n_x \times n_x}$ governs the temporal evolution of the latent state and characterizes the intrinsic dynamics of the neural population, and $C_y \in \mathbb{R}^{n_y \times n_x}$ is the observation matrix mapping latent states to neural activity.

This formulation intentionally excludes any measured external inputs. Because inputs are not modeled explicitly, the perturbation term w_k must absorb all influences not captured by the latent dynamics x_k, including both the effects of external inputs and any internally generated dynamics arising from the recorded region or from connected areas. However, by construction w_k is modeled as unstructured noise with no temporal dependencies across time steps. In many experimental settings, external inputs possess rich temporal structure (e.g., the trajectory of instructed movement targets or the time course of sensory stimuli). Consequently, any such persistent input-related structure observable in the neural activity cannot be carried by w_k and must instead be captured by the recurrent latent state x_k itself. The inferred latent state x_k, and therefore also the predicted neural activity $\hat{y}_k$, reflects a mixture of internally generated neural dynamics and externally driven temporal patterns, even though the latter originate outside the intrinsic mechanisms the model aims to characterize. This confounding complicates the interpretation of the latent states and limits the ability of NDM to isolate specific dynamical components, a fundamental limitation that motivates the use of input-driven dynamical models.

2.2 Input Non-preferential neural Dynamic Modeling

An extension of the classical non-preferential neural dynamic modeling framework is input-driven NDM (INDM) [3, 6], which explicitly incorporates measured inputs into the latent linear dynamical system. Unlike NDM, which must capture all temporal structure (including input dynamics) through the recurrent latent state x_k and its governing transition matrix A, INDM explicitly models the effect of measured inputs on both the latent state and neural activity. The INDM formulation is given by:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + w_k \\ y_k = C_y x_k + D_y u_k + v_k \end{cases}, \tag{2.2}$$

where $u_k \in \mathbb{R}^{n_u}$ represents the measured input to the neural population, such as inputs from other brain regions or sensory stimuli. The matrices $B \in \mathbb{R}^{n_x \times n_u}$ and $D_y \in \mathbb{R}^{n_y \times n_u}$ capture how these inputs drive the latent state and the observed neural activity, respectively. The remaining parameters are defined as in (2.1).

By explicitly representing measured inputs, INDM enables a clearer dissociation between externally driven and internally generated dynamics. The state transition matrix A in INDM reflects the intrinsic dynamics of the recorded brain region, rather than a mixture of intrinsic dynamics and input-driven patterns as in NDM, while the matrix B captures the input-driven contribution. The latent state x_k evolves according to both these intrinsic dynamics and the influence of measured inputs u_k and unmeasured excitations w_k.

However, INDM remains a non-preferential approach: it models all neural dynamics without distinguishing between components that are relevant to a specific behavior and those that are not. In other words, while INDM can dissociate intrinsic from input-driven dynamics, it does not allow for preferential modeling of dynamics shared between neural activity and behavior. This limitation motivates the development of methods that can combine explicit input modeling with the ability to prioritize and isolate behaviorally relevant dynamical components, such as IPSID.

2.3 Preferential Subspace Identification

While non-preferential methods like NDM or INDM model all temporal structure in neural activity without distinction, Preferential Subspace Identification (PSID) [1] addresses a different modeling challenge. In many experimental settings, the neural dynamics directly responsible for generating a specific behavior account for only a minor part of the recorded neural activity. The remaining neural activity reflects ongoing processes unrelated to the specific behavior under study, such as task-irrelevant sensory processing, or spontaneous neural fluctuations. PSID is designed to dissociate and preferentially learn these behaviorally relevant dynamics.

PSID formulates this preferential modeling problem through a structured state-space representation that jointly models neural activity y_k and behavior z_k:

$$\begin{cases} x_{k+1} = Ax_k + w_k, \\ y_k = C_y x_k + v_k, \\ z_k = C_z x_k + \epsilon_k \end{cases} \quad , \quad x_k = \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} \tag{2.3}$$

where the latent state x_k is partitioned into two orthogonal components: $x_k^{(1)} \in \mathbb{R}^{n_1}$ represents the behaviorally relevant states, while $x_k^{(2)} \in \mathbb{R}^{n_2}$ captures behaviorally irrelevant states that contribute to neural activity but not to the measured behavior. Here $C_z \in \mathbb{R}^{n_z \times n_x}$ is the behavior observation matrix and ϵ_k is the corresponding measurement noise.

The key structural constraint in PSID is that the behavior observation matrix takes the form

$$C_z = [C_{z_1}\ 0], \tag{2.4}$$

which enforces that only the behaviorally relevant states $x_k^{(1)}$ can influence the observed behavior z_k, while the behaviorally irrelevant states $x_k^{(2)}$ contribute exclusively to neural activity. The noise term ϵ_k represents behavioral dynamics that are not encoded in the recorded neural activity. The state transition matrix A, observation matrix C_y, and noise terms w_k and v_k are defined as in the standard NDM formulation.

PSID achieves this dissociation through a two-stage subspace identification procedure. In the first stage, behaviorally relevant states $x_k^{(1)}$ are identified with priority by extracting the subspace of neural activity that is most predictive of future behavior. This prioritized extraction ensures that model capacity (the available latent dimensions) is first dedicated to capturing behaviorally relevant dynamics. In an optional second stage, behaviorally irrelevant states $x_k^{(2)}$

are identified from the remaining neural activity, that is orthogonal to the behaviorally relevant subspace. This enables PSID to provide more accurate models of behaviorally relevant dynamics compared to non-preferential methods, which distribute model capacity across all sources of neural dynamics without distinction.

However, like NDM, PSID does not account for measured inputs to the neural system. The model assumes that all temporal structure in both neural activity and behavior arises from the latent dynamics x_k and noise terms. When external inputs possess temporal structure, PSID cannot distinguish whether observed temporal patterns in the behaviorally relevant dynamics originate from intrinsic mechanisms within the recorded brain region or simply reflect these external inputs. This limits the interpretability of the identified behaviorally relevant states and motivates the development of IPSID, which extends PSID's preferential modeling framework to explicitly incorporate measured inputs.

2.4 Input Preferential Subspace Identification

While PSID enables the dissociation and prioritized learning of behaviorally relevant dynamics, and INDM enables the explicit modeling of measured inputs, neither method addresses both challenges simultaneously. Input Preferential Subspace Identification (IPSID) [2] unifies these two capabilities, providing a framework that simultaneously dissociates intrinsic behaviorally relevant dynamics, intrinsic behaviorally irrelevant dynamics, input-driven dynamics and prioritizes the learning of the first.

IPSID addresses a fundamental confound that arises when both measured inputs and behavior exhibit temporal structure: without explicitly modeling inputs, it becomes impossible to determine whether observed correlations between neural activity and behavior reflect intrinsic neural computations within the recorded brain region or merely the common influence of external inputs on both signals. For example, in a reaching task where targets change over time, neural activity may correlate with hand kinematics simply because both are driven by the temporal structure of target trajectories, rather than because the recorded neurons intrinsically generate the movement. By incorporating measured inputs into the preferential modeling framework, IPSID can dissociate these input-driven correlations from the intrinsic behaviorally relevant dynamics that genuinely reflect neural computations underlying behavior.

IPSID formulates this problem through an input-output state-space representation that

jointly models neural activity y_k, behavior z_k, and measured inputs u_k:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + w_k, \\ y_k = C_y x_k + D_y u_k + v_k, \\ z_k = C_z x_k + D_z u_k + \epsilon_k \end{cases} \quad , \quad x_k = \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} \tag{2.5}$$

where, as in PSID, the latent state x_k is partitioned into behaviorally relevant states $x_k^{(1)} \in \mathbb{R}^{n_1}$ and behaviorally irrelevant states $x_k^{(2)} \in \mathbb{R}^{n_2}$. The measured input $u_k \in \mathbb{R}^{n_u}$ now explicitly appears in all three equations through the input matrices $B \in \mathbb{R}^{n_x \times n_u}$, $D_y \in \mathbb{R}^{n_y \times n_u}$, and $D_z \in \mathbb{R}^{n_z \times n_u}$. These matrices capture both the direct effects of inputs on neural activity and behavior, as well as their influence on the latent dynamics. As in PSID, the structural constraint $C_z = [C_{z_1}\ 0]$ ensures that only $x_k^{(1)}$ contributes to behavior through intrinsic dynamics, while $x_k^{(2)}$ affects only neural activity. The remaining parameters are defined as in the PSID formulation (2.3).

The key innovation in IPSID is a modified two-stage identification procedure that accounts for inputs while preserving the prioritized learning of behaviorally relevant dynamics. In the first stage, IPSID identifies the behaviorally relevant subspace by extracting neural temporal patterns that are predictive of future behavior after removing the effects of measured inputs. This ensures that $x_k^{(1)}$ captures intrinsic behaviorally relevant dynamics rather than apparent neural-behavioral correlations induced by common input drive. In the optional second stage, behaviorally irrelevant intrinsic dynamics $x_k^{(2)}$ are extracted from the remaining neural variance. Throughout both stages, the input-driven components (Bu_k, $D_y u_k$, $D_z u_k$) are identified jointly with the state dynamics, enabling IPSID to provide a complete dissociation of the three sources of temporal structure in neural population activity.

3 Preliminaries

In the previous section, we introduced several linear state-space models for neural dynamics, in particular the preferential subspace identification models PSID and IPSID. The identification procedures underlying these models rely on four key mathematical tools: oblique projections, singular value decomposition (SVD), block Hankel matrices, and Kalman filtering.

Oblique projections provide the flexibility to remove input contributions and decompose the residual neural activity into behaviorally relevant and irrelevant components in input-driven settings, while SVD reveals the low-dimensional structure needed to represent these components efficiently. Block Hankel matrices organize temporal data in a form that exposes dynamic relationships and enables efficient computation of these projections. Once the latent structure is identified through these geometric operations, and we have learned the system matrices, Kalman filtering provides the recursive algorithm for extracting states from neural observations.

Familiarity with these tools is essential to understand how preferential models identify and separate behaviorally relevant dynamics from recorded neural activity.

3.1 Oblique Projections

In subspace identification methods such as IPSID or PSID, it is often necessary to separate components of the data that lie in different subspaces (e.g., behaviorally relevant versus behaviorally irrelevant dynamics). While orthogonal projections can perform this separation when

the underlying subspaces are mutually orthogonal, they cannot cleanly distinguish different components when the subspaces overlap, a common scenario in neural data. In such cases, oblique projections [9] become essential, as they allow the extraction of distinct dynamical components even when the associated subspaces are not geometrically orthogonal and may partially overlap. This flexibility is crucial for the IPSID framework, which relies on oblique projections to dissociate behaviorally relevant intrinsic dynamics from other sources of neural variability.

Throughout this section, we assume that all matrices under consideration are wide, i.e., they have more columns than rows (or, more generally, sufficient column dimension), so that their row spaces are well defined and operators of the form $AA^\top$ are square. All projections in this section are taken with respect to row spaces.

Before introducing oblique projections, we recall the notion of orthogonal projections.

Definition 1 (Orthogonal Projection)**.** *An orthogonal projection of a matrix $Z \in \mathbb{R}^{m\times j}$ onto another matrix $Y \in \mathbb{R}^{n\times j}$ is defined as*

$$Z/Y \stackrel{\text{def}}{=} ZY^\top(YY^\top)^\dagger Y, \tag{3.1}$$

where the symbol † denotes the Moore-Penrose pseudoinverse.

The matrix $Y^\top(YY^\top)^\dagger Y$ acts as the orthogonal projection operator onto the row space of Y, therefore multiplying Z by this operator extracts the component of Z that lies in the row space of Y, i.e., the linear space spanned by the rows of Y. Equivalently, Z/Y solves the matrix least-squares problem

$$\min_A \|Z - AY\|_F^2. \tag{3.2}$$

Orthogonal projections naturally partition the data into independent contributions when subspaces are mutually orthogonal.

We now move on to the more general notion of oblique projections, which relaxes the orthogonality constraint [2, Appendix, Note S1]. To define oblique projections, we introduce the orthogonal complement projection operator Π_U, with respect to a matrix U:

$$\Pi_U \stackrel{\text{def}}{=} I - U^\top(UU^\top)^\dagger U. \tag{3.3}$$

This operator extracts the component of any matrix that is orthogonal to the row space of U. In other words, it removes the part that lies within the subspace spanned by the rows of U and

retains only the orthogonal residual. For any matrix Z, this gives:

$$Z\Pi_U = Z - Z/U. \tag{3.4}$$

With the orthogonal complement operator Π_U defined, we can present the notion of an oblique projection. The idea is to project a matrix Z onto the row space of another matrix Y, but along the row space of U rather than orthogonally. As before, we assume that Z, Y and U are wide matrices sharing a common column dimension.

Definition 2 (Oblique Projection)**.** *The oblique projection of a matrix $Z \in \mathbb{R}^{m\times j}$ onto another matrix $Y \in \mathbb{R}^{n\times j}$ along a matrix $U \in \mathbb{R}^{p\times j}$ is defined as*

$$Z/_U Y \overset{\text{def}}{=} (Z\Pi_U)(Y\Pi_U)^\top \left[(Y\Pi_U)(Y\Pi_U)^\top\right]^\dagger Y\Pi_U = (Z\Pi_U)/(Y\Pi_U). \tag{3.5}$$

This definition embodies a natural two-step procedure. First, $Z\Pi_U$ and $Y\Pi_U$ remove from both matrices any component lying in the row space of U, leaving only the parts orthogonal to U. The oblique projection then performs a standard orthogonal projection of the residual $Z\Pi_U$ onto the subspace spanned by the residual $Y\Pi_U$. This is made explicit in the right-hand side of (3.5), which shows that the oblique projection is simply an orthogonal projection applied to the deflated matrices.

Geometrically, this construction isolates the component of Z that is explained by Y but not already explained by U (Remark 1). This capability makes oblique projections indispensable for methods such as IPSID, where one must disentangle behaviorally relevant neural dynamics from other overlapping sources of variability. By explicitly accounting for the overlap through the auxiliary subspace U, oblique projections provide exactly the structured separation needed.

Remark 1 (Terminology)**.** *We say that Z is* explained by *Y if it lies in the row space of Y, i.e., if Z can be written as a linear combination of the rows of Y. This terminology is used consistently throughout the thesis.*

Illustrative Example. We show the difference between orthogonal and oblique projections with a simple numerical example. Consider the matrices

$$Z = \begin{bmatrix} 2 & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 1 \end{bmatrix}. \tag{3.6}$$

Here, Z represents a data vector containing contributions from two overlapping components, Y spans the subspace of interest, oriented along the horizontal direction, and U spans an aux-

iliary subspace along the diagonal, whose contribution we wish to remove.

The row spaces of Y and U are not orthogonal, since $YU^\top \neq 0$. The orthogonal projection of Z onto Y is given by

$$Z/Y = ZY^\top(YY^\top)^{-1}Y = \begin{bmatrix} 2 & 0 \end{bmatrix}. \tag{3.7}$$

This projection captures all components of Z lying in the row space of Y, including those that are also explained by U. As a result, the orthogonal projection cannot distinguish between contributions from Y and U when their subspaces overlap.

To isolate the component of Z that is predictable from Y but independent of U, we form the orthogonal complement projection

$$\Pi_U = I - U^\top(UU^\top)^{-1}U = \frac{1}{2}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \tag{3.8}$$

Applying this operator yields

$$Z\Pi_U = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}, \qquad Y\Pi_U = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}. \tag{3.9}$$

The oblique projection is then obtained as

$$Z/_U Y = (Z\Pi_U)/(Y\Pi_U) = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}. \tag{3.10}$$

Unlike the orthogonal projection, the oblique projection explicitly removes the component of Z lying in the row space of U before projecting onto Y, thereby isolating the component of Z that is predictable from Y but independent of U.

3.2 Singular Vector Decomposition

3.2.1 Existence of SVD

Singular value decomposition (SVD) is a matrix factorization that expresses any real or complex matrix as the product of three components: an initial rotation, a rescaling along orthogonal axes, and a final rotation [9]. More formally, SVD generalizes the eigendecomposition of square normal matrices (i.e., matrices with an orthonormal eigenbasis) to arbitrary $m \times n$ matrices.

Theorem 1 (Existence of SVD). *Any matrix $A \in \mathbb{R}^{m\times n}$, with $m \geq n$, can be decomposed as*

$$A = U \begin{bmatrix} \mathcal{D} \\ 0 \end{bmatrix} V^\top, \tag{3.11}$$

where $U = [u_1 \dots u_m] \in \mathbb{R}^{m\times m}$ and $V = [v_1 \dots v_n] \in \mathbb{R}^{n\times n}$ are orthogonal matrices and $\mathcal{D} = diag(\sigma_1, \dots \sigma_n)$ is a diagonal matrix whose entries satisfy

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n. \tag{3.12}$$

This theorem, [9, Theorem 2.4.1], establishes the existence of the singular value decomposition in (3.11) for any real matrix. The result extends naturally to complex matrices by replacing orthogonal matrices with unitary ones.

In the presented decomposition, the diagonal entries of the matrix $\mathcal{D}$, denoted $\sigma_1, \dots, \sigma_n$, are the *singular values* of A, and they are uniquely determined. They quantify how strongly the matrix stretches vectors along specific directions. Because the singular values are ordered from largest to smallest in the decomposition, they indicate the relative importance of each mode, with larger singular values corresponding to directions of greater variance in the data. Additionally, the number of non-zero singular values equals the rank of A, providing a direct connection between the algebraic and geometric properties of the matrix.

The columns of V are the *right singular vectors*, which define these input directions, while the columns of U are the *left singular vectors*, which define the corresponding output directions. Both sets of singular vectors form orthonormal bases. Unlike singular values, singular vectors are not always uniquely determined. When a singular value is simple (i.e., not repeated), its corresponding singular vectors are unique up to a change of sign. When singular values are repeated, the associated singular subspace is uniquely defined, but any orthonormal basis can be chosen within that subspace.

The assumption $m \geq n$ of Theorem 1 is not restrictive. If A is an $m \times n$ matrix with $m < n$, the theorem can be applied to its transpose $A^\top$, yielding

$$A^\top = U \begin{bmatrix} \mathcal{D} \\ 0 \end{bmatrix} V^\top, \tag{3.13}$$

Therefore, the singular value decomposition of A is

$$A = V \begin{bmatrix} \mathcal{D} & 0 \end{bmatrix} U^\top. \tag{3.14}$$

Furthermore, if $m = n$ we can write the SVD of the square matrix A more compactly as

$$A = U\mathcal{D}V^\top. \tag{3.15}$$

3.2.2 Truncated SVD

An important property of SVD is that the singular values provide a natural measure of the energy or variance captured by each mode. Consequently, in many applications, including neural data analysis, most of the meaningful structure in a matrix is concentrated in the first few singular values, while the remaining ones capture noise or low-energy fluctuations. By retaining only the largest k singular values and their corresponding singular vectors, we obtain the rank-k truncated SVD:

$$A_k \stackrel{\text{def}}{=} U_k \mathcal{D}_k V_k^\top, \tag{3.16}$$

where $U_k = [u_1 \dots u_k]$, $V_k = [v_1 \dots v_k]$ and $\mathcal{D}_k = \text{diag}(\sigma_1, \dots \sigma_k)$.

By the Eckart-Young-Mirsky theorem, [9, Theorem 2.4.8], A_k is the best rank-k approximation of A with respect to both the Frobenius norm and the spectral norm.

Definition 3 (Frobenius norm)**.** *The Frobenius norm of a matrix $A \in \mathbb{R}^{m\times n}$ is defined as the square root of the sum of the squared entries of A:*

$$\|A\|_F \stackrel{\text{def}}{=} \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2}.$$

Definition 4 (Spectral norm)**.** *The spectral norm of a matrix $A \in \mathbb{R}^{m\times n}$ is defined as the induced ℓ_2 operator norm, i.e.,*

$$\|A\|_2 \stackrel{\text{def}}{=} \max_{x\neq 0} \frac{\|Ax\|_2}{\|x\|_2}.$$

Equivalently, $\|A\|_2$ is equal to the largest singular value of A, and can be expressed as

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^\top A)}.$$

Theorem 2 (Eckart-Young-Mirsky – spectral norm version)**.** *Let $A \in \mathbb{R}^{m \times n}$ have rank r with $r > k \geq 1$. Then, the matrix minimization problem*

$$\min_{Z \,:\, rank(Z)=k} \|A - Z\|_2 \tag{3.17}$$

has the solution

$$Z = A_k = U_k \mathcal{D}_k V_k^\top, \tag{3.18}$$

and the value of the minimum corresponds to

$$\|A - A_k\|_2 = \sigma_{k+1}. \tag{3.19}$$

A similar version of Theorem 2 exists for the Frobenius norm. These results make truncated SVD a standard tool for dimensionality reduction, denoising, and the extraction of low-dimensional structure, which are essential capabilities in modeling high-dimensional neural recordings.

In practice, selecting the truncation rank k involves examining the spectrum of singular values. Common approaches include setting a threshold based on the cumulative variance explained (e.g., retaining enough modes to capture 90% of the total variance $\sum_{i=1}^{k} \sigma_i^2$). In neural data analysis, the appropriate choice of k depends on the modeling objective: strong compression favors lower k, denoising exploits spectral gaps between signal and noise, and latent-structure identification requires retaining modes tied to behaviorally or physiologically meaningful dynamics.

3.3 Block Hankel Matrices

Block Hankel matrices [3] provide a structured way to organize time-series data that facilitates the analysis of underlying system dynamics. This construction enables a compact representation of the system dynamics over finite time horizons and forms the basis for subspace-based identification.

Definition and Construction

Given a sequence of measurements $\{s_k : 0 \leq k < N\}$, with $s_k \in \mathbb{R}^{n_s}$, and a projection horizon i, we define the block Hankel matrix S:

$$S \stackrel{\text{def}}{=} \begin{bmatrix} s_0 & s_1 & \cdots & s_{j-1} \\ s_1 & s_2 & \cdots & s_j \\ \vdots & \vdots & \ddots & \vdots \\ s_{2i-1} & s_{2i} & \cdots & s_{j+2i-2} \end{bmatrix}. \tag{3.20}$$

The structure of S exhibits the characteristic Hankel property: each column contains $2i$ consecutive measurements, with successive columns representing a temporal shift of one time step. Note that each entry s_k is itself a column vector in $\mathbb{R}^{n_s}$, so each column of S is a vector in $\mathbb{R}^{2in_s}$ obtained by vertically concatenating $2i$ such vectors, giving S a total of $2in_s$ rows and j columns. In the preferential modeling framework we set $j = N - 2i + 1$, to ensure that all available measurements are used.

Past-Future Decomposition

The matrix S admits a natural decomposition into past and future components:

$$S = \begin{bmatrix} S_p \\ S_f \end{bmatrix} = \begin{bmatrix} s_0 & s_1 & \cdots & s_{j-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_{i-1} & s_i & \cdots & s_{j+i-2} \\ s_i & s_{i+1} & \cdots & s_{j+i-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_{2i-1} & s_{2i} & \cdots & s_{j+2i-2} \end{bmatrix}. \tag{3.21}$$

Here $S_p, S_f \in \mathbb{R}^{in_s \times j}$, where S_p denotes the block comprising rows 0 through $i-1$ and collects the past measurements over a horizon of length i, while S_f denotes the block comprising rows i through $2i-1$ and collects the corresponding future measurements.

An alternative partitioning of S provides shifted past and future blocks, defined as:

$$S = \begin{bmatrix} S_p^+ \\ S_f^- \end{bmatrix} = \begin{bmatrix} s_0 & s_1 & \cdots & s_{j-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_i & s_{i+1} & \cdots & s_{j+i-1} \\ s_{i+1} & s_{i+2} & \cdots & s_{j+i} \\ \vdots & \vdots & \ddots & \vdots \\ s_{2i-1} & s_{2i} & \cdots & s_{j+2i-2} \end{bmatrix}, \tag{3.22}$$

where $S_p^+ \in \mathbb{R}^{(i+1)n_s \times j}$ is the extended past block consisting of rows 0 through i, and $S_f^- \in \mathbb{R}^{(i-1)n_s \times j}$ is the reduced future block consisting of rows $i+1$ through $2i-1$.

For notational convenience, we denote the ℓ-th block row of the Hankel matrix by:

$$S_\ell \stackrel{\text{def}}{=} \begin{bmatrix} s_\ell & s_{\ell+1} & \cdots & s_{\ell+j-1} \end{bmatrix} \in \mathbb{R}^{n_s \times j}. \tag{3.23}$$

In particular, S_i is the block row at the horizon i.

3.4 Kalman Filters

The Kalman filter is a recursive algorithm for estimating the hidden state of a linear dynamical system observed through noisy measurements. Originally developed by Rudolf Kalman in 1960 [10], it has become one of the most widely used state estimation techniques across engineering, robotics, navigation, and neuroscience. The filter's popularity stems from its optimality properties: under the assumptions of linear dynamics, Gaussian noise, and known system parameters, the Kalman filter provides the minimum mean-squared-error estimate of the state at each time step.

Within the preferential modeling framework, the Kalman filter serves as the key mechanism for recovering the latent neural states from new data after the model parameters have been identified [3, 6]. The Kalman filter derivations presented in this section apply to both IPSID and PSID. For PSID, the formulation simplifies by setting the input matrices $B = 0$, $D_y = 0$ and $D_z = 0$, which eliminates all input-related terms from the state evolution and observation equations. The resulting filter operates identically but without input contributions.

3.4.1 Time-Varying Kalman Filter

The IPSID state-space model, introduced in Section 2.4, represents the evolution of neural activity, behavior, and inputs over time as follows:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + w_k \\ y_k = C_y x_k + D_y u_k + v_k \\ z_k = C_z x_k + D_z u_k + \epsilon_k \end{cases}, \tag{3.24}$$

with x_k low-dimensional latent state representing the underlying neural dynamics, u_k measured input signal (e.g., sensory or task instruction inputs), y_k recorded neural activity and z_k measured behavior. The process noise w_k and measurement noise v_k are modeled as zero-mean, temporally white, Gaussian random variables that are independent of x_k. Their joint covariance structure is given by:

$$\mathbb{E}\left\{ \begin{bmatrix} w_k \\ v_k \end{bmatrix} \begin{bmatrix} w_k \\ v_k \end{bmatrix}^T \right\} = \begin{bmatrix} Q & S \\ S^T & R \end{bmatrix} \tag{3.25}$$

where Q and R are the process and measurement noise covariance matrices, respectively, and S captures any cross-correlation between them. Finally, ϵ_k is a Gaussian random process representing the residual dynamics.

For the purpose of state estimation, we focus on the state evolution and neural observation equations:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + w_k \\ y_k = C_y x_k + D_y u_k + v_k \end{cases}. \tag{3.26}$$

Assuming that the model parameters (A, B, C_y, D_y) and noise covariances (Q, R, S) have been identified, the Kalman filter provides the optimal recursive estimator for the latent state x_k based on the observed neural activity y_k and inputs u_k [11]. Notably, behavioral measurements z_k are not used during this estimation phase.

The filter operates through a two-stage prediction-correction cycle that optimally balances the information from the system dynamics and the new measurements at each time step k.

Prediction step: Using the system dynamics, the filter propagates the state estimate and its

uncertainty forward in time:

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_{k-1}, \tag{3.27}$$

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q, \tag{3.28}$$

where $\hat{x}_{k|k-1}$ denotes the predicted state at time k given observations up to time $k-1$ and the matrix $P_{k|k-1}$ represents the predicted error covariance, defined as

$$P_{k|k-1} \overset{\text{def}}{=} \mathbb{E}\{(x_k - \hat{x}_{k|k-1})(x_k - \hat{x}_{k|k-1})^\top\}, \tag{3.29}$$

which quantifies the uncertainty in the state prediction.

Update step: Upon receiving the new observation y_k, the filter refines the prediction:

$$K_k = (AP_{k-1|k-1}C_y^T + S)(C_yP_{k|k-1}C_y^T + R)^{-1}, \tag{3.30}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C_y\hat{x}_{k|k-1} - D_yu_k), \tag{3.31}$$

$$P_{k|k} = (I - K_kC_y)P_{k|k-1}, \tag{3.32}$$

where $\hat{x}_{k|k}$ is the updated state estimate incorporating the measurement y_k, and K_k is the Kalman gain that optimally weights the innovation $e_k \overset{\text{def}}{=} y_k - C_y\hat{x}_{k|k-1} - D_yu_k$ based on the relative uncertainties in the prediction and measurement.

The Kalman gain determines the optimal weighting between the model-based prediction and the measurement. When measurement noise dominates ($R \gg C_yP_{k|k-1}C_y^\top$), reflecting unreliable observations, the gain becomes small and the filter relies primarily on the model prediction. Conversely, when measurement noise is negligible ($R \ll C_yP_{k|k-1}C_y^\top$), the gain increases and the filter places greater weight on the measurement.

3.4.2 Steady-State Kalman Filter

While the time-varying Kalman filter is optimal at each time step, its computational cost can be prohibitive for processing long neural recordings. Fortunately, under mild conditions, the error covariance P_k and Kalman gain K_k converge to constant steady-state values, enabling a computationally efficient implementation.

Convergence to Steady-State

We assume that the identified models constitute *minimal realizations*, meaning they use the smallest possible state dimension n_x to capture the observed dynamics of y_k. In what follows, we work under steady-state conditions, meaning that transient effects from initial conditions have decayed and all covariance quantities are time-invariant.

For such minimal realizations, the convergence of P_k to a steady-state value P_∞ is guaranteed when the system satisfies:

- **Observability:** The pair (A, C_y) is observable, meaning all state components (or equivalently, all dynamical modes of A) can be reconstructed from the output measurements. This ensures observations provide sufficient information to track the entire state.

- **Reachability:** The pair (A, G_y), with

$$G_y \stackrel{\text{def}}{=} \mathbb{E}\{x_{k+1} y_k^\top\}, \tag{3.33}$$

  is reachable, meaning all state components are influenced by the process noise through the dynamics. This ensures the error covariance remains bounded.

Let us define the neural observability matrix Γ_y and the neural reachability matrix Δ_y as:

$$\Gamma_y \stackrel{\text{def}}{=} \begin{bmatrix} C_y \\ C_y A \\ \vdots \\ C_y A^{n_x-1} \end{bmatrix}, \quad \Delta_y \stackrel{\text{def}}{=} \begin{bmatrix} G_y & AG_y & \cdots & A^{n_x-1} G_y \end{bmatrix}. \tag{3.34}$$

Observability is satisfied when Γ_y has full column rank n_x [6, Theorem 3.4]. Reachability is satisfied when Δ_y has full row rank n_x [6, Theorem 3.7]. In both cases, full rank is a sufficient and necessary condition. For neural data applications, these conditions are generally satisfied: neural recordings provide rich observations of underlying dynamics (ensuring observability), and process noise captures the stochastic variability inherent in neural activity (ensuring reachability).

Once P_k converges, the Kalman gain stabilizes to a constant value K_∞, and the filter dynamics become time-invariant [11].

Steady-State Solution

The steady-state error covariance P_∞ can be computed directly by solving the Discrete Algebraic Riccati Equation (DARE) [11], obtained by setting $P_{k+1|k} = P_{k|k-1} = P$ in the covariance recursion:

$$P = APA^T + Q - (APC_y^T + S)(C_yPC_y^T + R)^{-1}(APC_y^T + S)^T. \tag{3.35}$$

Once P_∞ is obtained by solving (3.35), the steady-state Kalman gain is computed as:

$$K_\infty = (AP_\infty C_y^T + S)(C_yP_\infty C_y^T + R)^{-1}. \tag{3.36}$$

Steady-State Filter

With K_∞ precomputed, the steady-state Kalman filter reduces to a simple time-invariant recursion that combines the prediction and update steps:

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k-1} + Bu_k + K_\infty(y_k - C_y\hat{x}_{k|k-1} - D_yu_k). \tag{3.37}$$

For PSID, where $B = 0$ and $D_y = 0$, this simplifies to:

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k-1} + K_\infty(y_k - C_y\hat{x}_{k|k-1}). \tag{3.38}$$

4

Preferential Subspace Identification

Preferential Subspace Identification (PSID) [1] is a dynamical systems framework designed to dissociate neural population activity into behaviorally relevant dynamics and behaviorally irrelevant dynamics. Unlike conventional approaches that fail to prioritize behaviorally relevant activity, PSID introduces a preferential learning mechanism that explicitly focuses model capacity on identifying the intrinsic neural dynamics that drive behavior.

The method is based on the following state-space formulation, introduced in Chapter 2:

$$\begin{cases} x_{k+1} = Ax_k + w_k, \\ y_k = C_y x_k + v_k, \\ z_k = C_z x_k + \epsilon_k \end{cases}, \quad x_k = \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix}, \tag{4.1}$$

where $y_k \in \mathbb{R}^{n_y}$ denotes the observed neural activity, $z_k \in \mathbb{R}^{n_z}$ denotes the measured behavior, and the latent state $x_k \in \mathbb{R}^{n_x}$ is partitioned into behaviorally relevant states $x_k^{(1)} \in \mathbb{R}^{n_1}$ and behaviorally irrelevant states $x_k^{(2)} \in \mathbb{R}^{n_2}$. The process noise $w_k \in \mathbb{R}^{n_x}$ and measurement noise $v_k \in \mathbb{R}^{n_y}$ are zero-mean white noise sequences, independent of the latent state x_k, with joint covariance

$$\mathbb{E}\left\{ \begin{bmatrix} w_k \\ v_k \end{bmatrix} \begin{bmatrix} w_k \\ v_k \end{bmatrix}^\top \right\} = \begin{bmatrix} Q & S \\ S^\top & R \end{bmatrix}. \tag{4.2}$$

The term $\epsilon_k \in \mathbb{R}^{n_z}$ represents the component of behavior not generated by the latent state x_k,

therefore absent from the recorded neural activity, and is assumed to be zero-mean and independent of x_k, w_k, and v_k, with no further assumptions on its dynamics.

This system admits an equivalent formulation [1, 6] that makes the separation between behaviorally relevant and irrelevant dynamics explicit:

$$\begin{cases} \begin{bmatrix} x_{k+1}^{(1)} \\ x_{k+1}^{(2)} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + \begin{bmatrix} w_k^{(1)} \\ w_k^{(2)} \end{bmatrix} \\ y_k = \begin{bmatrix} C_{y_1} & C_{y_2} \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + v_k \\ z_k = \begin{bmatrix} C_{z_1} & 0 \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + \epsilon_k \end{cases} . \tag{4.3}$$

Note that $A_{12} = 0$ ensures that the evolution of the behaviorally relevant states $x_k^{(1)}$ is not influenced by the behaviorally irrelevant states $x_k^{(2)}$, while $C_{z_2} = 0$ ensures only $x_k^{(1)}$ contributes to the measured behavior z_k.

When behaviorally relevant dynamics represent only a fraction of recorded neural activity, standard identification methods may fail to capture them accurately. PSID addresses this limitation through a two-stage procedure that prioritizes the extraction of dynamics predictive of behavior before modeling remaining neural activity. Given neural recordings $\{y_k : 0 \leq k < N\}$, behavioral measurements $\{z_k : 0 \leq k < N\}$, chosen dimensions n_1 (behaviorally relevant) and $n_2 = n_x - n_1$ (behaviorally irrelevant), and i projection horizon, the PSID algorithm proceeds as follows:

- **Stage 1:** Extract the n_1-dimensional subspace $x_k^{(1)}$ containing dynamics most predictive of future behavior.
- **Stage 2:** Extract the n_2-dimensional subspace $x_k^{(2)}$ from the remaining neural activity, orthogonal to $x_k^{(1)}$.
- **Final Stage:** Identify the system parameters.

Stage 2 is optional and can be omitted if only behaviorally relevant dynamics are of interest. If $n_1 = 0$, Stage 1 is skipped entirely and the algorithm simplifies to standard non-preferential subspace identification applied in Stage 2 [3]. The following sections present each stage in detail.

Primary and Secondary Signal

Before introducing the algorithm, we briefly clarify the roles of the observed signals. In the formulation in (4.1), neural activity y_k serves as the *primary signal*, meaning that the latent state x_k is constructed to describe the complete dynamics of y_k, including both its shared dynamics with behavior and its behaviorally irrelevant dynamics. Behavioral measurements z_k serve as the *secondary signal*: while z_k shares dynamics with y_k through the common latent state x_k, the state does not fully describe the dynamics of z_k. The secondary signal instead acts as a supervisory signal that guides which aspects of neural dynamics are captured in the behaviorally relevant subspace $x_k^{(1)}$.

In particular, the term $C_z x_k$ in the behavioral equation represents the shared dynamics, while the term ϵ_k captures any dynamics of behavior that are not shared with the primary signal. If modeling these unshared behavioral dynamics is also of interest, PSID can be extended accordingly.

The designation of primary and secondary signals is in principle interchangeable: the choice determines only which signal's dynamics are fully described beyond the shared dynamics. In this work, we take neural activity as the primary signal because our goal is to decompose neural dynamics based on their relevance to behavior.

4.1 System-Theoretic Matrices

Before proceeding to the PSID algorithm outline, it is useful to collect several structured matrices that arise naturally from the state-space model and appear repeatedly throughout the subspace identification framework [3]. We assume throughout that the system is stable and that all covariance quantities are time-invariant, as guaranteed under steady-state conditions.

Covariance matrices. The steady-state second-order statistics of the system are characterized by three covariance matrices: the state covariance Σ_x, the neural output covariance Σ_y, the cross-covariance G_y between the next latent state and the current neural output, defined as

$$\Sigma_x \stackrel{\text{def}}{=} \mathbf{E}[x_k x_k^\top], \tag{4.4}$$

$$\Sigma_y \stackrel{\text{def}}{=} \mathbf{E}[y_k y_k^\top], \tag{4.5}$$

$$G_y \stackrel{\text{def}}{=} \mathbf{E}[x_{k+1} y_k^\top]. \tag{4.6}$$

These covariances are related to the model noise statistics via

$$\Sigma_x = A\Sigma_x A^\top + Q, \tag{4.7}$$
$$\Sigma_y = C_y\Sigma_x C_y^\top + R, \tag{4.8}$$
$$G_y = A\Sigma_x C_y^\top + S. \tag{4.9}$$

Consequently, the system (4.1) can be equivalently characterized in terms of $(\Sigma_x, \Sigma_y, G_y)$ rather than the noise covariances (Q, R, S).

This reparameterization is central to the PSID algorithm, which expresses the dynamics directly in terms of observable covariances. Indeed, Faurre's stochastic realization theorem [3, Theorem 5] shows that infinitely many triplets (Q, R, S) can generate the same second-order output statistics, making them individually non-identifiable from data. Conversely, Σ_y, and G_y are uniquely determined by the output second-order statistics, up to a similarity transform. This is why the parameterization $(\Sigma_x, \Sigma_y, G_y)$ is the natural one for PSID, which expresses the system dynamics directly in terms of these observable covariances.

Observability matrices. The neural observability matrix $\Gamma_y \in \mathbb{R}^{n_x n_y \times n_x}$ and the behavior observability matrix $\Gamma_z \in \mathbb{R}^{n_x n_z \times n_x}$ are defined as

$$\Gamma_y \overset{\text{def}}{=} \begin{bmatrix} C_y \\ C_y A \\ \vdots \\ C_y A^{n_x-1} \end{bmatrix}, \qquad \Gamma_z \overset{\text{def}}{=} \begin{bmatrix} C_z \\ C_z A \\ \vdots \\ C_z A^{n_x-1} \end{bmatrix}. \tag{4.10}$$

These matrices describe how the latent state evolves forward in time and becomes observable through the neural and behavioral output sequences, respectively.

As discussed in Section 3.4.2, we assume that the pair (A, C_y) is observable, guaranteeing that Γ_y has full column rank. However, the pair (A, C_z) does not need to be observable, and consequently Γ_z may be rank-deficient. In particular, if $n_1 < n_x$, with n_1 dimension of the behaviorally relevant latent states $x_k^{(1)}$, then the behavior observability matrix has exactly n_1

non-zero columns and can be written as

$$\Gamma_z = \begin{bmatrix} C_{z_1} & 0 \\ C_{z_1}A_{11} & 0 \\ \vdots & \vdots \\ C_{z_1}A_{11}^{n_x-1} & 0 \end{bmatrix} = \begin{bmatrix} \Gamma_z^{(1)} & 0 \end{bmatrix}. \tag{4.11}$$

Note that, for some analyses, it is convenient to consider the k-step truncated version of the observability matrices:

$$\Gamma_{y_k} \stackrel{\text{def}}{=} \begin{bmatrix} C_y \\ C_yA \\ \vdots \\ C_yA^{k-1} \end{bmatrix}, \qquad \Gamma_{z_k} \stackrel{\text{def}}{=} \begin{bmatrix} C_z \\ C_zA \\ \vdots \\ C_zA^{k-1} \end{bmatrix}. \tag{4.12}$$

Reachability matrix. The neural reachability matrix $\Delta_y \in \mathbb{R}^{n_x \times n_x n_y}$ is defined as

$$\Delta_y \stackrel{\text{def}}{=} \begin{bmatrix} G_y & AG_y & \cdots & A^{n_x-1}G_y \end{bmatrix}. \tag{4.13}$$

This matrix characterizes how stochastic excitation propagates through and accumulates in the latent state over n_x time steps. For certain analyses it is also useful to work with the reversed neural reachability matrix,

$$\Delta_y^c \stackrel{\text{def}}{=} \begin{bmatrix} A^{n_x-1}G_y & \cdots & AG_y & G_y \end{bmatrix}, \tag{4.14}$$

in which the columns appear in reverse chronological order, as well as its k-step version

$$\Delta_{y_k}^c \stackrel{\text{def}}{=} \begin{bmatrix} A^{k-1}G_y & \cdots & AG_y & G_y \end{bmatrix}. \tag{4.15}$$

4.2 Algorithm Outline

Given the training time series of neural activity $\{y_k : 0 \leq k < N\}$, measured behavior $\{z_k : 0 \leq k < N\}$, along with user-specified state dimension n_x, dimensions $n_1 \leq n_x$ and $n_2 = n_x - n_1$, and projection horizon $i \geq 2$, the PSID identification algorithm proceeds in two stages. Throughout this section, we assume that both $n_1 > 0$ and $n_2 > 0$.

Stage 1

Consider the wide matrices Y_p, Y_f, Y_p^+, Y_f^- and Z_p, Z_f, Z_p^+, Z_f^-, as defined in Section 3.3.

1. Compute the least squares prediction of future behavior Z_f from past neural activity Y_p:

$$\hat{Z}_f = Z_f / Y_p. \tag{4.16}$$

2. Compute the singular value decomposition (SVD) (see Section 3.2) of $\hat{Z}_f$ and retain the top n_1 singular values, to identify the subspace most predictive of behavior:

$$\hat{Z}_f = U \mathcal{D} V^\top \approx U_1 \mathcal{D}_1 V_1^\top. \tag{4.17}$$

3. Compute the behavioral observability matrix $\Gamma_{z_i}^{(1)}$:

$$\Gamma_{z_i}^{(1)} = U_1 \mathcal{D}_1^{1/2} \tag{4.18}$$

and the behaviorally relevant latent state sequence $\hat{X}_i^{(1)}$:

$$\hat{X}_i^{(1)} = \Gamma_{z_i}^{(1)\dagger} \hat{Z}_f = \left(U_1 \mathcal{D}_1^{1/2} \right)^\dagger \left(Z_f / Y_p \right). \tag{4.19}$$

4. Repeat at the shifted projection horizon $i+1$. Compute:

$$\hat{Z}_f^- = Z_f^- / Y_p^+, \tag{4.20}$$

extract the reduced behavioral observability matrix by removing the last n_z rows of $\Gamma_{z_i}^{(1)}$:

$$\Gamma_{z_{i-1}}^{(1)} = \Gamma_{z_i}^{(1)}{}_{[1:(i-1)n_z,:]}, \tag{4.21}$$

and compute the shifted latent state sequence:

$$\hat{X}_{i+1}^{(1)} = \Gamma_{z_{i-1}}^{(1)}{}^\dagger \hat{Z}_f^- = \left(U_1 \mathcal{D}_1^{1/2} \right)^\dagger |_{rows\ 1:(i-1)n_z} \left(Z_f^- / Y_p^+ \right). \tag{4.22}$$

Stage 2

Stage 2 follows a procedure analogous to Stage 1, but operates on the residual neural activity rather than the full neural signal. Specifically, it extracts behaviorally irrelevant dynamics from

the remaining neural data after removing the contribution of the behaviorally relevant states identified in Stage 1.

5. Find the neural observability matrix $\Gamma_{y_i}^{(1)}$ for $\hat{X}_i^{(1)}$ as the least squares predictor of future neural activity Y_f from behaviorally relevant states $\hat{X}_i^{(1)}$:

$$\Gamma_{y_i}^{(1)} = Y_f \hat{X}_i^{(1)\top} \left(\hat{X}_i^{(1)} \hat{X}_i^{(1)\top} \right)^{\dagger}, \tag{4.23}$$

and subtract the corresponding prediction from Y_f:

$$Y_f' = Y_f - \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)} = Y_f - Y_f / \hat{X}_i^{(1)}. \tag{4.24}$$

This operation removes the neural activity already explained by the behaviorally relevant states $\hat{X}_i^{(1)}$ from Y_f.

6. Compute the least squares prediction of residual future neural activity Y_f' from past neural activity Y_p:

$$\hat{Y}_f' = Y_f' / Y_p. \tag{4.25}$$

7. Compute the SVD of $\hat{Y}_f'$ and retain the top n_2 singular values, to identify the remaining neural subspace:

$$\hat{Y}_f' = U' \mathcal{D}' V'^{\top} \approx U_2 \mathcal{D}_2 V_2^{\top}. \tag{4.26}$$

8. Compute the residual neural observability matrix $\Gamma_{y_i}^{(2)}$:

$$\Gamma_{y_i}^{(2)} = U_2 \mathcal{D}_2^{1/2} \tag{4.27}$$

and the behaviorally irrelevant latent state sequence $\hat{X}_i^{(2)}$:

$$\hat{X}_i^{(2)} = \Gamma_{y_i}^{(2)\dagger} \hat{Y}_f' = \left(U_2 \mathcal{D}_2^{1/2} \right)^{\dagger} \left(Y_f' / Y_p \right) \tag{4.28}$$

9. Repeat at the shifted projection horizon $i+1$. Compute the reduced neural observability matrix by removing the last n_y rows of $\Gamma_{y_i}^{(1)}$:

$$\Gamma_{y_{i-1}}^{(1)} = \Gamma_{y_i}^{(1)}{}_{[1:(i-1)n_y,:]}, \tag{4.29}$$

and subtract the corresponding predicted contribution from Y_f^-:

$$Y_f^{-'} = Y_f^- - \Gamma_{y_{i-1}}^{(1)} \hat{X}_{i+1}^{(1)} = Y_f^- - Y_f^- / \hat{X}_{i+1}^{(1)}. \tag{4.30}$$

Next, compute:

$$\hat{Y}_f^{-'} = Y_f^{-'} / Y_p^+, \tag{4.31}$$

and form the reduced residual neural observability matrix by removing the last n_y rows of $\Gamma_{y_i}^{(2)}$:

$$\Gamma_{y_{i-1}}^{(2)} = \Gamma_{y_i}^{(2)}{}_{[1:(i-1)n_y,:]}. \tag{4.32}$$

Finally, estimate the shifted latent state sequence as:

$$\hat{X}_{i+1}^{(2)} = \Gamma_{y_{i-1}}^{(2)}{}^{\dagger} \hat{Y}_f^{-'} = \left(U_2 \mathcal{D}_2^{1/2} \right)^{\dagger} |_{rows\ 1:(i-1)n_y} \left(Y_f^{-'} / Y_p^+ \right). \tag{4.33}$$

Final Stage

Having extracted all latent state components at horizons i and $i+1$, the algorithm proceeds to its final stage: parameter identification.

10. Concatenate the behaviorally relevant and irrelevant state components to construct the full latent state sequences:

$$\hat{X}_i = \begin{bmatrix} \hat{X}_i^{(1)} \\ \hat{X}_i^{(2)} \end{bmatrix}, \quad \hat{X}_{i+1} = \begin{bmatrix} \hat{X}_{i+1}^{(1)} \\ \hat{X}_{i+1}^{(2)} \end{bmatrix}. \tag{4.34}$$

11. Estimate the behaviorally relevant dynamics matrix via least squares:

$$A_{11} = \hat{X}_{i+1}^{(1)} \hat{X}_i^{(1)\dagger}. \tag{4.35}$$

Estimate the remaining dynamics matrices using the full latent state:

$$\begin{bmatrix} A_{21} & A_{22} \end{bmatrix} = \hat{X}_{i+1}^{(2)} \hat{X}_i^{\dagger}, \tag{4.36}$$

and assemble the full dynamics matrix:

$$A = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}. \tag{4.37}$$

12. Estimate the observation matrices via least squares using the latent states and the observations:

$$C_y = Y_i \hat{X}_i^\dagger, \quad C_z = Z_i \hat{X}_i^\dagger. \tag{4.38}$$

13. Compute the residuals:

$$W_i = \hat{X}_{i+1} - A\hat{X}_i, \quad V_i = Y_i - C_y \hat{X}_i \tag{4.39}$$

14. Estimate the process and observation noise covariances using the sample covariance of the residuals:

$$\begin{bmatrix} Q & S \\ S^\top & R \end{bmatrix} = \frac{1}{j} \begin{bmatrix} W_i \\ V_i \end{bmatrix} \begin{bmatrix} W_i \\ V_i \end{bmatrix}^\top. \tag{4.40}$$

This concludes the learning phase of the algorithm, having identified all model parameters A, C_y, C_z, Q, R, and S.

Simplified Cases

The algorithm handles edge cases naturally by omitting stages that extract absent state components. When $n_1 = 0$, Stage 1 is skipped and Stage 2 operates on the full neural activity Y_f, reducing to standard non-preferential identification [3]. When $n_2 = 0$, only behaviorally relevant dynamics are extracted in Stage 1, and Stage 2 is omitted. In both cases, parameter identification in the final stage uses only the extracted state components:

- If $n_1 = 0$ (no behaviorally relevant states): set $\hat{X}_i = \hat{X}_i^{(2)}$ and $\hat{X}_{i+1} = \hat{X}_{i+1}^{(2)}$.
- If $n_2 = 0$ (no behaviorally irrelevant states): set $\hat{X}_i = \hat{X}_i^{(1)}$ and $\hat{X}_{i+1} = \hat{X}_{i+1}^{(1)}$.

4.3 Algorithm Details

Having outlined the structure of the PSID algorithm in Section 4.2, we now present the technical details of each stage. The key to making the identification procedure tractable is the forward

innovation formulation, an equivalent reformulation of the state-space model that enables direct extraction of latent states through geometric projections. We begin by deriving this formulation and explaining why it resolves the circular dependence and noise correlation issues inherent in the original system (4.1). Before proceeding to the full algorithm, we also establish the least-squares framework for parameter identification, which clarifies how system matrices are recovered once the latent states are available. With these foundations in place, we then present the detailed algorithmic steps for state extraction and parameter identification.

4.3.1 Forward Innovation Formulation

Given the linear state-space formulation of equation (4.1) with known parameters, the Kalman filter yields optimal predictions in the minimum mean-square error sense (Section 3.4.2). Under the assumption of Gaussian noise, the best predictions of neural activity $\hat{y}_{k+1|k}$ and behavior $\hat{z}_{k+1|k}$ based on past observations $y_1, \ldots, y_k$ and $z_1, \ldots, z_k$ are computed recursively as

$$\begin{cases} \hat{x}_{k+1|k} = A\hat{x}_{k|k-1} + K_k(y_k - C_y\hat{x}_{k|k-1}) \\ \hat{y}_{k+1|k} = C_y\hat{x}_{k+1|k} \\ \hat{z}_{k+1|k} = C_z\hat{x}_{k+1|k} \end{cases}, \tag{4.41}$$

where K_k is the time-varying Kalman gain. The term

$$e_k \stackrel{\text{def}}{=} y_k - C_y\hat{x}_{k|k-1} \tag{4.42}$$

is the innovation, representing the component of current neural activity that cannot be predicted from past observations.

However, when system parameters are unknown, learning them from data using the original formulation (4.1) presents significant challenges. The fundamental issue is that the latent states x_k are unobserved, creating a circular dependence: extracting states requires knowing the parameters, yet learning parameters requires access to the states. Moreover:

- the process noise w_k and measurement noise v_k can be correlated, with cross-covariance S as defined in (4.2);

- the measurement noise v_k directly confounds observations of the state dynamics, preventing clean separation of state contributions from noise.

This correlation structure prevents the use of geometric methods: when projecting future observations onto past data, the correlated noise terms do not vanish, precluding the clean extraction of the state-space structure through linear algebra operations.

To overcome these issues, we reformulate the system. By assuming the Kalman filter has reached steady-state with gain $K = K_\infty$ (3.36), we can rewrite equation (4.1) in the forward innovation form:

$$\begin{cases} x_{k+1} = Ax_k + Ke_k \\ y_k = C_y x_k + e_k \\ z_k = C_z x_k + \epsilon_k \end{cases}, \tag{4.43}$$

where, for notational convenience, we define $x_k \overset{\text{def}}{=} \hat{x}_{k|k-1}$ and e_k is the innovation process defined in (4.42). Note that equations (4.1) and (4.43) are equivalent descriptions of the same system dynamics, they only differ in their internal state representations and noise processes.

This reformulation resolves each of the previously mentioned challenges.

First, it rewrites the noise structure in a more efficient way, consolidating the three noise covariance matrices (Q, R, S), that are individually non-identifiable from data (Section 4.1), into the single Kalman gain matrix K. Moreover, it establishes a direct relationship between the noise in the latent states and the noise in the neural activity: both are driven by the same innovation process e_k (scaled by K for the states). This means we only need K and e_k to fully characterize the noise structure. Furthermore, the innovation e_k is white noise uncorrelated with all past observations, transforming the noise from a confounding factor into a well-behaved stochastic process.

Second, thanks to this change, e_k vanishes under projection operations onto past data, allowing projections of future outputs onto past observations to cleanly eliminate all noise terms. This property is crucial because it reveals two fundamental linear relationships in the system:

1. The optimal estimate of the latent state x_k given past data can be expressed as a linear combination of past neural observations y_k.

2. The optimal predictions of future neural activity y_k and behavior z_k are linear functions of the current state estimate x_k.

Together, these relationships enable the use of tractable geometric methods that extract latent states directly from data through projection operations, without requiring knowledge of the system parameters. This reformulation thus breaks the circular dependence: states can be ex-

tracted first through geometric projections, and once extracted, the model parameters can be learned in a subsequent step, yielding a two-stage identification procedure.

Explicit Formulation of State Linear Relationship

The state linear relationship can be made explicit through recursive substitution of the state equation. By iterating the forward innovation formulation (4.43) backwards from time k, the predicted latent state can be expressed as a linear combination of past neural observations and the initial state estimate. For the case of zero initial conditions ($\hat{x}_0 = 0$ and $P_0 = 0$), this relationship is formalized as Theorem 6 in [3, p. 69]:

$$x_k = \Delta^c_{y_k} \Lambda^{-1}_{y_k} \begin{bmatrix} y_0 \\ \vdots \\ y_{k-1} \end{bmatrix}, \tag{4.44}$$

where $\Delta^c_{y_k}$ is defined as in (4.15), and Λ_{y_k} is the block covariance matrix

$$\Lambda_{y_k} \stackrel{\text{def}}{=} \begin{bmatrix} \Sigma_{y_0} & \Sigma_{y_{-1}} & \cdots & \Sigma_{y_{-(k-1)}} \\ \Sigma_{y_1} & \Sigma_{y_0} & \cdots & \Sigma_{y_{-(k-2)}} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{y_{k-1}} & \Sigma_{y_{k-1}} & \cdots & \Sigma_{y_0} \end{bmatrix}, \tag{4.45}$$

with the notation $\Sigma_{y_d} \stackrel{\text{def}}{=} \mathbb{E}\{y_{k+d} y_k^\top\}$. Moreover, these covariances admit the closed-form expressions

$$\Sigma_{y_d} = C_y A^{d-1} G_y, \qquad \Sigma_{y_{-d}} = (C_y A^{d-1} G_y)^\top, \tag{4.46}$$

which shows that the state x_k depends only on A, Σ_y, C_y, G_y and is linear in y_k [1, Appendix].

4.3.2 Least Squares Parameter Estimate

PSID estimates the system matrices in (4.3) via least-squares regression applied to block Hankel matrices constructed from the observed time series. In the following, we detail the estimation procedure for the sub-block A_{11} of the state transition matrix A. The remaining sub-blocks and other system matrices can be obtained analogously by applying the same methodology to the appropriate system equations.

Problem Formulation

We consider the state dynamics associated with the behaviorally relevant latent subspace. For a generic instant k, the state evolution is governed by:

$$x_k^{(1)} = A_{11} x_{k-1}^{(1)} + w_{k-1}^{(1)}. \tag{4.47}$$

This equation describes how the latent state evolves from one time step to the next as a function of its current value and process noise. While this single-step relationship holds at any time instant, estimating A_{11} from a single observation results in an underdetermined problem. Therefore, PSID leverages multiple consecutive state observations organized into block Hankel matrices (introduced in Section 3.3).

Let $\hat{X}_i^{(1)}$ and $\hat{X}_{i+1}^{(1)}$ denote the estimated behaviorally relevant latent states at two consecutive time horizons, structured as:

$$\begin{aligned} \hat{X}_i^{(1)} &= \begin{bmatrix} \hat{x}_i^{(1)} & \hat{x}_{i+1}^{(1)} & \cdots & \hat{x}_{i+j-1}^{(1)} \end{bmatrix} \in \mathbb{R}^{n_1 \times j} \\ \hat{X}_{i+1}^{(1)} &= \begin{bmatrix} \hat{x}_{i+1}^{(1)} & \hat{x}_{i+2}^{(1)} & \cdots & \hat{x}_{i+j}^{(1)} \end{bmatrix} \in \mathbb{R}^{n_1 \times j} \end{aligned}, \tag{4.48}$$

where $j = N - 2i + 1$ is the number of temporal windows determined by the data length N and projection horizon i. Applying equation (4.47) to each time instant in the window yields a sequence of state equations:

$$\begin{aligned} \hat{x}_{i+1}^{(1)} &= A_{11} \hat{x}_i^{(1)} + w_i^{(1)}, \\ \hat{x}_{i+2}^{(1)} &= A_{11} \hat{x}_{i+1}^{(1)} + w_{i+1}^{(1)}, \\ &\vdots \\ \hat{x}_{i+j}^{(1)} &= A_{11} \hat{x}_{i+j-1}^{(1)} + w_{i+j-1}^{(1)}. \end{aligned} \tag{4.49}$$

By stacking these equations column-wise into their respective matrices, we obtain the compact representation:

$$\hat{X}_{i+1}^{(1)} = A_{11} \hat{X}_i^{(1)} + W^{(1)}, \tag{4.50}$$

where $W^{(1)}$ collects the stacked process noise.

Equation (4.50) is a linear matrix regression in which the parameter matrix Θ and regressor matrix Φ are:

$$\Theta \overset{\text{def}}{=} A_{11} \in \mathbb{R}^{n_1 \times n_1}, \qquad \Phi \overset{\text{def}}{=} \hat{X}_i^{(1)} \in \mathbb{R}^{n_1 \times j}, \tag{4.51}$$

where Θ contains all unknown parameters to be estimated, and Φ contains all known (or previously estimated) quantities. The problem then takes the standard form:

$$\hat{X}_{i+1}^{(1)} = \Theta\Phi + W^{(1)}. \tag{4.52}$$

Provided that the number of columns satisfies $j \gg n_1$ and that Φ has full row rank, the regression problem in (4.52) is overdetermined and admits a least-squares solution [9]. This enables reliable estimation of the unknown matrix A_{11} via least-squares regression despite the presence of measurement noise.

Least Squares Solution

To estimate Θ from the noisy observations in equation (4.52), we minimize the sum of squared residuals:

$$\hat{\Theta} = \arg\min_{\Theta} \left\| \hat{X}_{i+1}^{(1)} - \Theta\Phi \right\|_F^2 \tag{4.53}$$

where$\| \cdot \|_F$ denotes the Frobenius norm introduced in Section 3.2. This problem is a classical linear least-squares estimation and admits a closed-form solution. Under the assumption that Φ has full row rank, the unique least-squares solution is

$$\hat{\Theta} = \hat{X}_{i+1}^{(1)} \Phi^\top \left(\Phi\Phi^\top \right)^{-1} = \hat{X}_{i+1}^{(1)} \Phi^\dagger. \tag{4.54}$$

Substituting back the definitions of Θ and Φ, we obtain the following estimate:

$$A_{11} = \hat{X}_{i+1}^{(1)} \hat{X}_i^{(1)\dagger}. \tag{4.55}$$

This completes the estimation of A_{11}. The remaining system matrices are estimated analogously where applicable, with exceptions detailed in the sections that follow. With this framework in place, we now move on to the full algorithmic procedure.

4.3.3 Stage 1: Extracting Behaviorally Relevant States

Step 1. The forward innovation formulation establishes two key linear relationships. First, the latent state x_k is a linear combination of past neural observations y_k, as shown in (4.44). Second, the predictable component of behavior z_k is linear in the latent state x_k, as given by the third equation in (4.43). Combining these two relationships leads to the central result underlying the first stage of PSID: the predictable component of behavior can be expressed as

a linear function of past neural activity, and therefore lies in the subspace spanned by the past observations y_k. This structure allows the behaviorally relevant latent states to be recovered through orthogonal projection operations, without prior knowledge of the model parameters. In particular, projecting future behavior z_k onto past neural activity y_k isolates the behaviorally relevant dynamics, removing both measurement noise v_k and behaviorally irrelevant neural components. This is the reasoning behind equation (4.16).

In more detail, at each time step k we consider the previous i samples as the past and the next i samples as the future, with i the given projection horizon. Using the notation defined in Section 3.3, let Y_p denote past neural activity, Y_f future neural activity, Z_p past behavior, and Z_f future behavior. We project future behavior onto past neural activity to extract the predictable component of behavior, obtaining equation (4.16):

$$\hat{Z}_f = Z_f / Y_p = Z_f Y_p^\top (Y_p Y_p^\top)^{-1} Y_p. \tag{4.56}$$

Steps 2–3. Observe that $Z_f Y_p^\top$ is the cross-covariance between past neural activity and future behavior, and $Y_p Y_p^\top$ is the covariance of past neural activity. These population covariances can be estimated via their sample covariance counterparts:

$$\Sigma_{z_f y_p} \overset{\text{def}}{=} \frac{1}{j} Z_f Y_p^\top, \qquad \Sigma_{y_p y_p} \overset{\text{def}}{=} \frac{1}{j} Y_p Y_p^\top, \tag{4.57}$$

where $j = N - 2i + 1$ is the number of data columns in the Hankel matrices. Then, the least squares solution of equation (4.16) can be written as:

$$\hat{Z}_f = \Sigma_{z_f y_p} \Sigma_{y_p y_p}^{-1} Y_p. \tag{4.58}$$

Note that the sample covariance estimates are asymptotically unbiased and converge to their population counterparts as $j \to \infty$. To characterize this convergence, we introduce the block covariance matrix:

$$\Lambda_{zy_i} \overset{\text{def}}{=} \begin{bmatrix} \Sigma_{zy_i} & \Sigma_{zy_{i-1}} & \cdots & \Sigma_{zy_1} \\ \Sigma_{zy_{i+1}} & \Sigma_{zy_i} & \cdots & \Sigma_{zy_2} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{zy_{2i-1}} & \Sigma_{zy_{2i-2}} & \cdots & \Sigma_{zy_i} \end{bmatrix}, \tag{4.59}$$

where $\Sigma_{zy_d} \overset{\text{def}}{=} \mathbb{E}\{z_{k+d} y_k^\top\}$ denotes the cross-covariance between behavior at time $k+d$ and neural activity at time k. It can be shown that as $j \to \infty$, the sample covariances converge to

these block covariance matrices:

$$\Sigma_{z_f y_p} \to \Lambda_{z y_i}, \qquad \Sigma_{y_p y_p} \to \Lambda_{y_i}, \tag{4.60}$$

where Λ_{y_i} is defined in (4.45).

As for Σ_{y_d} in (4.45), equivalent relationships hold for $\Sigma_{z y_d}$:

$$\Sigma_{z y_d} = C_z A^{d-1} G_y, \qquad \Sigma_{z y_{-d}} = (C_z A^{d-1} G_y)^\top. \tag{4.61}$$

Substituting these expressions into the definition of $\Lambda_{z y_i}$ yields:

$$\Lambda_{z y_i} = \begin{bmatrix} C_z \\ C_z A \\ \vdots \\ C_z A^{i-1} \end{bmatrix} \begin{bmatrix} A^{i-1} G_y & A^{i-2} G_y & \cdots & A G_y & G_y \end{bmatrix}. \tag{4.62}$$

The two matrices on the right-hand side are the behavior observability matrix Γ_{z_i} introduced in (4.12) and the reversed reachability matrix $\Delta^c_{y_i}$ introduced in (4.15), respectively, giving:

$$\Lambda_{z y_i} = \Gamma_{z_i} \Delta^c_{y_i}. \tag{4.63}$$

Putting these results together and recalling the linear relationship between states and neural activity (4.44), in the limit $j \to \infty$, equation (4.16) becomes:

$$\hat{Z}_f = \Sigma_{z_f y_p} \Sigma^{-1}_{y_p y_p} Y_p = \Lambda_{z y_i} \Lambda^{-1}_{y_i} Y_p = \Gamma_{z_i} \Delta^c_{y_i} \Lambda^{-1}_{y_i} Y_p = \Gamma_{z_i} \hat{X}_i, \tag{4.64}$$

where $\hat{X}_i$ is the matrix of Kalman state estimates, with each column computed from the past i observations of y_k. Here, we have successively used (4.58), (4.60), (4.63), and (4.44).

Taking this one step further, we observe that the behavior observability matrix Γ_{z_i} is not necessarily full rank and has exactly n_1 non-zero columns, as shown in (4.11). Therefore,

$$\hat{Z}_f = \Gamma_{z_i} \hat{X}_i = \begin{bmatrix} \Gamma^{(1)}_{z_i} & 0 \end{bmatrix} \begin{bmatrix} \hat{X}^{(1)}_i \\ \hat{X}^{(2)}_i \end{bmatrix} = \Gamma^{(1)}_{z_i} \hat{X}^{(1)}_i, \tag{4.65}$$

where $\hat{X}^{(1)}_i$ denotes the first n_1 rows of $\hat{X}_i$, representing the behaviorally relevant states, and $\hat{X}^{(2)}_i$ denotes the remaining rows, representing the behaviorally irrelevant states. Note that the

predicted behavior $\hat{Z}_f$ is only dictated by the behaviorally relevant states $\hat{X}_i^{(1)}$.

Since both $\Gamma_{z_i}^{(1)}$ and $\hat{X}_i^{(1)}$ are full rank with rank n_1, in the limit $j \to \infty$ the row space of $\hat{Z}_f$ equals the row space of $\hat{X}_i^{(1)}$ exactly. This enables extraction of behaviorally relevant states directly from $\hat{Z}_f$ via singular value decomposition, offering two advantages: no prior knowledge of model parameters is required, and behaviorally relevant states are prioritized while deferring irrelevant components to a subsequent stage.

To implement this extraction, we compute the singular value decomposition of $\hat{Z}_f$ and retain the top n_1 singular values:

$$\hat{Z}_f = U\mathcal{D}V^\top \approx U_1\mathcal{D}_1V_1^\top. \tag{4.66}$$

Asymptotically, $\hat{Z}_f$ has exactly n_1 non-zero singular values. With finite data, we retain the top n_1 singular values to extract the behaviorally relevant subspace while suppressing noise in the remaining singular values. The factorization in (4.65) then yields:

$$\Gamma_{z_i}^{(1)} = U_1\mathcal{D}_1^{1/2}, \qquad \hat{X}_i^{(1)} = \mathcal{D}_1^{1/2}V_1^\top, \tag{4.67}$$

corresponding to equations (4.17) and (4.18) in the algorithm. Finally, the behaviorally relevant states can also be extracted via pseudoinverse as in (4.19).

Step 4. Following the same derivation as in Steps 1–3, we obtain analogous formulas at the shifted time horizon $i+1$ by replacing Y_p, Y_f with Y_p^+, Y_f^-, and Z_p, Z_f with Z_p^+, Z_f^-. Specifically, projecting the shifted future behavior onto shifted past neural activity yields:

$$\hat{Z}_f^- = Z_f^-/Y_p^+ = Z_f^-{Y_p^+}^\top(Y_p^+{Y_p^+}^\top)^{-1}Y_p^+. \tag{4.68}$$

Applying the same asymptotic analysis and factorization as before, we obtain:

$$\hat{Z}_f^- = \Gamma_{z_{i-1}}\Delta_{y_{i+1}}^c\Lambda_{y_{i+1}}^{-1}Y_p^+ = \Gamma_{z_{i-1}}^{(1)}\hat{X}_{i+1}^{(1)}, \tag{4.69}$$

where $\Gamma_{z_{i-1}}$, $\Delta_{y_{i+1}}^c$, $\Lambda_{y_{i+1}}$, and $\Gamma_{z_{i-1}}^{(1)}$ are defined analogously to their horizon-i counterparts, the columns of $\hat{X}_{i+1}$ are Kalman state estimates computed from the past $i+1$ observations of y_k, and $\hat{X}_{i+1}^{(1)}$ denotes the first n_1 rows of $\hat{X}_{i+1}$ representing the behaviorally relevant states.

By definition of $\Gamma_{z_{i-1}}^{(1)}$, we can obtain it directly from $\Gamma_{z_i}^{(1)}$ computed in the previous step by removing the last block row. This observation allows us to extract $\hat{X}_{i+1}^{(1)}$ efficiently as shown in

(4.22).

4.3.4 Stage 2: Extracting Behaviorally Irrelevant States

Before starting the derivation of Stage 2, let us make an observation: following the same steps as in Stage 1 for the projection of Y_f onto Y_p, we get

$$\hat{Y}_f = \Sigma_{y_f y_p} \Sigma_{y_p y_p}^{-1} Y_p = \Gamma_{y_i} \Delta_{y_i}^c \Lambda_{y_i}^{-1} Y_p = \Gamma_{y_i} \hat{X}_i = \begin{bmatrix} \Gamma_{y_i}^{(1)} & \Gamma_{y_i}^{(2)} \end{bmatrix} \begin{bmatrix} \hat{X}_i^{(1)} \\ \hat{X}_i^{(2)} \end{bmatrix}, \tag{4.70}$$

where the neural observability matrix Γ_{y_i}, the neural reachability matrix $\Delta_{y_i}^c$, and the block covariance matrix Λ_{y_i} are defined in (4.12), (4.15), and (4.45), respectively, and $\hat{X}_i$, $\hat{X}_i^{(1)}$, $\hat{X}_i^{(2)}$ as in (4.64). Here, $\Gamma_{y_i}^{(1)}$ denotes the first n_1 columns of Γ_{y_i}, $\Gamma_{y_i}^{(2)}$ the remaining ones, consistently with the partitioning of $\hat{X}_i$ into $\hat{X}_i^{(1)}$ and $\hat{X}_i^{(2)}$.

In contrast to $\hat{Z}_f$, which depends only on $\hat{X}_i^{(1)}$, the row space of $\hat{Y}_f$ is the union of the row spaces of both the behaviorally relevant states $\hat{X}_i^{(1)}$ and the behaviorally irrelevant states $\hat{X}_i^{(2)}$. This is the key distinction that motivates Stage 2. The derivation that follows exploits this property: we isolate the behaviorally irrelevant component from the neural predictions by subtracting out the contribution from the behaviorally relevant subspace already identified in Stage 1. The remaining residuals then contain only the behaviorally irrelevant component, from which we extract $\hat{X}_i^{(2)}$ via SVD.

Step 5. Having extracted the behaviorally relevant states $\hat{X}_i^{(1)}$ in Stage 1, we now turn to the remaining neural dynamics. If the behaviorally irrelevant states $\hat{X}_i^{(2)}$ are also of interest, we first have to isolate the component of Y_f not explained by $\hat{X}_i^{(1)}$. To do so, we remove from Y_f any contribution lying in the subspace spanned by $\hat{X}_i^{(1)}$.

Since (4.70) holds asymptotically, we have

$$\hat{Y}_f = \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)} + \Gamma_{y_i}^{(2)} \hat{X}_i^{(2)}. \tag{4.71}$$

As the model parameters A and C_y are not yet known, $\Gamma_{y_i}^{(1)}$ cannot be computed directly from its definition. Instead, exploiting the asymptotic relation above, we can estimate it from data as the least squares solution for predicting Y_f from $\hat{X}_i^{(1)}$:

$$\Gamma_{y_i}^{(1)} = Y_f \hat{X}_i^{(1)\top} \left(\hat{X}_i^{(1)} \hat{X}_i^{(1)\top} \right)^{\dagger}. \tag{4.72}$$

This recovers the true $\Gamma_{y_i}^{(1)}$ asymptotically because $\hat{X}_i^{(1)}$ and $\hat{X}_i^{(2)}$ are uncorrelated. We then obtain the residual future neural activity as in (4.24):

$$Y_f' = Y_f - \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)}, \tag{4.73}$$

which retains only the component of Y_f lying outside the subspace spanned by $\hat{X}_i^{(1)}$. By construction, Y_f' contains contributions from the behaviorally irrelevant states $\hat{X}_i^{(2)}$ only.

Step 6. The same reasoning that underlies Stage 1 applies here: the forward innovation formulation guarantees that $x_k^{(2)}$ is a linear combination of past neural observations y_k. Moreover, by construction, the residual neural activity y_k' retains only the contribution of $x_k^{(2)}$, and is therefore itself linear in past y_k. It follows that y_k' lies in the subspace spanned by past observations, and projecting Y_f' onto Y_p isolates the behaviorally irrelevant dynamics $x_k^{(2)}$. This justifies (4.25).

Steps 7–8. From (4.70), it holds that

$$\hat{Y}_f = \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)} + \Gamma_{y_i}^{(2)} \hat{X}_i^{(2)}. \tag{4.74}$$

By definition of Y_f' and linearity of the projection operator, we can write

$$\hat{Y}_f' = Y_f'/Y_p = Y_f/Y_p - \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)}/Y_p \tag{4.75}$$

Since $\hat{X}_i^{(1)}$ is a linear function of Y_p, the term $\Gamma_{y_i}^{(1)} \hat{X}_i^{(1)}$ already lies in the row space of Y_p, and projecting it onto Y_p leaves it unchanged: $\Gamma_{y_i}^{(1)} \hat{X}_i^{(1)}/Y_p = \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)}$. Substituting and applying (4.70), it follows that asymptotically

$$\hat{Y}_f' = \hat{Y}_f - \Gamma_{y_i}^{(1)} \hat{X}_i^{(1)} = \Gamma_{y_i}^{(2)} \hat{X}_i^{(2)}. \tag{4.76}$$

As $\Gamma_{y_i}^{(2)}$ and $\hat{X}_i^{(2)}$ are both full rank of rank $n_2 = n_x - n_1$, the row space of $\hat{Y}_f'$ coincides (asymptotically) with that of $\hat{X}_i^{(2)}$. This justifies applying to Y_f' a procedure entirely analogous to the SVD-based extraction carried out in Stage 1, now targeting the remaining n_2 behaviorally irrelevant states. We therefore apply SVD to $\hat{Y}_f'$:

$$\hat{Y}_f' = U' \mathcal{D}' V'^{\top} \approx U_2 \mathcal{D}_2 V_2^{\top}, \tag{4.77}$$

retaining the top n_2 singular values to prioritize the directions of greatest unexplained neural variance. Thanks to the decomposition established in (4.76), the residual neural observability matrix is then retrieved as in (4.27) and the behaviorally irrelevant state sequence is obtained as in (4.28).

STEP 9. Following the same logic as Step 5–8, we repeat the extraction at the shifted projection horizon $i+1$. The shifted residual $Y_f^{-'}$ is obtained by subtracting from Y_f^- the contribution of the behaviorally relevant states $\hat{X}_{i+1}^{(1)}$, using the reduced observability matrix $\Gamma_{y_{i-1}}^{(1)}$ obtained by removing the last n_y rows of $\Gamma_{y_i}^{(1)}$. Analogously, $\Gamma_{y_{i-1}}^{(2)}$ is obtained by removing the last n_y rows of $\Gamma_{y_i}^{(2)}$, without requiring a new SVD. The shifted behaviorally irrelevant states $\hat{X}_{i+1}^{(2)}$ are then extracted via pseudoinverse as in (4.33).

4.3.5 FINAL STAGE: PARAMETER IDENTIFICATION

Having recovered all latent states, we proceed to the identification of the system matrices.

STEPS 10–12. We construct the full latent state by concatenating the behaviorally relevant and behaviorally irrelevant latent states, as in (4.34). We then estimate the sub-blocks of the state matrix A defined in (4.3), namely A_{11}, A_{21}, A_{22}, and the observation matrices C_y and C_z via least squares from the extracted state sequences and the observed data, exploiting the fact that $\hat{X}_i$ and $\hat{X}_{i+1}$ serve as sample estimates of the true latent states x_k and x_{k+1}, respectively. This follows the same procedure described in Section 4.3.2 and yields the formulas in (4.35)–(4.38).

STEPS 13–14. Once the system matrices A and C_y have been identified, we estimate the noise sequences W_i and V_i as the residuals of the state and output equations in (4.1):

$$w_k = x_{k+1} - Ax_k, \qquad v_k = y_k - C_y x_k, \tag{4.78}$$

which quantify the portion of the state evolution and neural activity not explained by the identified deterministic model. Since the true latent states x_k are not directly observed, we replace them with their estimates $\hat{X}_i$ and $\hat{X}_{i+1}$, and y_k with the observed data matrix Y_i, yielding (4.39).

The noise covariance matrices Q, S, and R are then estimated as the sample covariances of these residuals, as in (4.40). This concludes the parameter identification phase.

Remark 2 (Alternative estimation of C_z)**.** *A more refined approach for estimating C_z is to first run the Kalman filter on the training data y_k using the already identified parameters A, C_y, Q, R, and S to obtain the filtered state estimates $\hat{x}_{k+1|k}$ for each time step k. Stacking these into a matrix $\hat{X}_{k+1|k}$, we then estimate C_z via least squares:*

$$C_z = Z_k \hat{X}_{k+1|k}^{\dagger}, \tag{4.79}$$

where Z_k is the matrix of stacked behavior observations defined in (3.23)*. Since C_z does not enter the Kalman filter recursions, it can be estimated separately in this second pass without affecting latent state extraction. This approach is adopted in the original PSID paper [1] and yields a more accurate estimate of C_z since the Kalman-filtered states $\hat{x}_{k+1|k}$ are more accurate estimates of the true latent states, having been obtained using all past observations $y_0, y_1, \ldots, y_k$, whereas the subspace-extracted states $\hat{X}_i$ rely only on the past i observations.*

5

Input Preferential Subspace Identification

Input Preferential Subspace Identification (IPSID) [2] extends PSID to settings where external inputs are explicitly modelled. While PSID dissociates neural population activity into behaviorally relevant and behaviorally irrelevant intrinsic dynamics, it does not account for the influence of external inputs in its model. As a consequence, when inputs are present in the data, PSID cannot dissociate intrinsic dynamics from input-driven dynamics, and the temporal structure of the input may be incorrectly attributed to intrinsic neural dynamics. IPSID addresses this limitation by explicitly incorporating input-driven dynamics alongside intrinsic ones, while preserving the preferential learning mechanism that prioritizes the identification of the neural dynamics that drive behavior. By jointly modelling intrinsic and input-driven dynamics and their interactions, IPSID provides a more accurate way to isolate the low-dimensional neural processes that directly support behavior from the surrounding high-dimensional activity.

The IPSID model augments the PSID state-space formulation (4.1) with explicit input terms:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + w_k, \\ y_k = C_y x_k + D_y u_k + v_k, \\ z_k = C_z x_k + D_z u_k + \epsilon_k \end{cases} \quad , \quad x_k = \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix}, \tag{5.1}$$

where $u_k \in \mathbb{R}^{n_u}$ denotes the external input and all remaining parameters are defined as in (4.1).

Writing out the block structure explicitly, the state-space model takes the equivalent form [2, 6]:

$$\begin{cases} \begin{bmatrix} x_{k+1}^{(1)} \\ x_{k+1}^{(2)} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + \begin{bmatrix} B_1 \\ B_2 \end{bmatrix} u_k + \begin{bmatrix} w_k^{(1)} \\ w_k^{(2)} \end{bmatrix} \\ y_k = \begin{bmatrix} C_{y_1} & C_{y_2} \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + D_y u_k + v_k \\ z_k = \begin{bmatrix} C_{z_1} & 0 \end{bmatrix} \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix} + D_z u_k + \epsilon_k \end{cases} . \tag{5.2}$$

The block structure is identical to that of PSID in (4.3), with $A_{12} = 0$ and $C_{z_2} = 0$, augmented by the input terms Bu_k, $D_y u_k$, and $D_z u_k$.

Just as PSID, also IPSID follows a two-stage procedure, extending each projection to remove the contribution of future inputs. Given neural recordings $\{y_k : 0 \leq k < N\}$, behavioral measurements $\{z_k : 0 \leq k < N\}$, inputs $\{u_k : 0 \leq k < N\}$, chosen dimensions n_1 (behaviorally relevant) and $n_2 = n_x - n_1$ (behaviorally irrelevant), and projection horizon i, IPSID proceeds as follows:

- **Stage 1:** Extract the n_1-dimensional subspace $x_k^{(1)}$ of intrinsic neural dynamics most predictive of future behavior, via an oblique projection that removes the contribution of future inputs.
- **Stage 2:** Extract the n_2-dimensional subspace $x_k^{(2)}$ of remaining intrinsic neural dynamics from the residual activity after subtracting the influence of both $x_k^{(1)}$ and future inputs.
- **Final Stage:** Identify the system parameters.

As in PSID, Stage 2 is optional. Setting $n_1 = 0$ skips Stage 1 entirely, reducing IPSID to INDM [3]. The following sections present each stage in detail.

Primary and Secondary Signal

In the IPSID formulation (5.1), the roles of neural activity y_k and behavior z_k as primary and secondary signals are preserved unchanged from PSID. The input u_k is neither: it is a fully observed external influence that enters the model directly through the terms Bu_k, $D_y u_k$, and $D_z u_k$, rather than being absorbed into the latent state. As a consequence, x_k captures only the

intrinsic dynamics of the recorded region, which is precisely the dissociation that motivates IPSID.

5.1 System-Theoretic Matrices

Before proceeding, we collect the definitions of structured matrices that appear repeatedly throughout the derivation [3]. As in PSID, we assume the system is stable and operating in steady state, so that all covariance quantities are time-invariant.

Deterministic reachability matrix. Parallel to the stochastic neural reachability matrix Δ_y introduced in Section 4.1, the deterministic reachability matrix $\Delta_u \in \mathbb{R}^{n_x \times n_x n_u}$ is defined as

$$\Delta_u \stackrel{\text{def}}{=} \begin{bmatrix} B & AB & \cdots & A^{n_x-1}B \end{bmatrix}. \tag{5.3}$$

This matrix characterizes how inputs propagate through and accumulate in the latent state over n_x time steps. For the identification procedure it is more useful to work with the reversed deterministic reachability matrix,

$$\Delta_u^c \stackrel{\text{def}}{=} \begin{bmatrix} A^{n_x-1}B & \cdots & AB & B \end{bmatrix}, \tag{5.4}$$

in which the columns appear in reverse chronological order, as well as its k-step version

$$\Delta_{u_k}^c \stackrel{\text{def}}{=} \begin{bmatrix} A^{k-1}B & \cdots & AB & B \end{bmatrix}. \tag{5.5}$$

Deterministic Markov parameter matrices. The matrices $H_{y_k} \in \mathbb{R}^{kn_y \times kn_u}$ and $H_{z_k} \in \mathbb{R}^{kn_z \times kn_u}$ are the lower block-triangular Toeplitz matrices of deterministic Markov parameters for the neural and behavior outputs respectively:

$$H_{y_k} = \begin{bmatrix} D_y & 0 & \cdots & 0 \\ C_yB & D_y & \cdots & 0 \\ C_yAB & C_yB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ C_yA^{k-2}B & C_yA^{k-3}B & \cdots & D_y \end{bmatrix}, \tag{5.6}$$

$$H_{z_k} = \begin{bmatrix} D_z & 0 & \cdots & 0 \\ C_z B & D_z & \cdots & 0 \\ C_z AB & C_z B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ C_z A^{k-2} B & C_z A^{k-3} B & \cdots & D_z \end{bmatrix}. \tag{5.7}$$

These matrices capture the causal input-output impulse response: they map a sequence of inputs to their contribution to the respective outputs through the system's Markov parameters. The lower triangular structure reflects causality, as the output at time k can only be influenced by inputs up to and including time k.

5.2 Algorithm Outline

The IPSID algorithm takes as input training time series of neural activity $\{y_k : 0 \leq k < N\}$, behavior $\{z_k : 0 \leq k < N\}$, and input $\{u_k : 0 \leq k < N\}$, together with user-specified parameters $n_x, n_1 \leq n_x, n_2 = n_x - n_1$, and projection horizon $i \geq 2$. We assume throughout that $n_1 > 0$ and $n_2 > 0$.

Stage 1

We consider the wide matrices Y_p, Y_f, Y_p^+, Y_f^-, Z_p, Z_f, Z_p^+, Z_f^-, and U_p, U_f, U_p^+, U_f^- as defined in Section 3.3.

1. Compute the prediction of future behavior Z_f from past inputs U_p and past neural activity Y_p, via an oblique projection (see Section 3.1) that removes the contribution of future inputs U_f:

$$\hat{Z}_f = Z_f /_{U_f} \begin{bmatrix} U_p \\ Y_p \end{bmatrix} \tag{5.8}$$

2. Compute the SVD of $\hat{Z}_f$ (see Section 3.2) and keep the top n_1 singular values, to identify the subspace most predictive of behavior:

$$\hat{Z}_f = U \mathcal{D} V^\top \approx U_1 \mathcal{D}_1 V_1^\top \tag{5.9}$$

3. Extract the behavioral observability matrix $\Gamma_{z_i}^{(1)}$:

$$\Gamma_{z_i}^{(1)} = U_1 \mathcal{D}_1^{1/2}, \tag{5.10}$$

and the behaviorally relevant latent state sequence $\hat{X}_i^{(1)}$ via pseudoinverse:

$$\hat{X}_i^{(1)} = {\Gamma_{z_i}^{(1)}}^{\dagger} \left(Z_f / \begin{bmatrix} U_p \\ Y_p \\ U_f \end{bmatrix} \right). \tag{5.11}$$

4. Repeat at the shifted projection horizon $i+1$. Compute the shifted oblique projection:

$$\hat{Z}_f^- = Z_f^- /_{U_f^-} \begin{bmatrix} U_p^+ \\ Y_p^+ \end{bmatrix}, \tag{5.12}$$

remove the last n_z rows of $\Gamma_{z_i}^{(1)}$ to obtain the reduced observability matrix:

$$\Gamma_{z_{i-1}}^{(1)} = \Gamma_{z_i\ [1:(i-1)n_z,:]}^{(1)} = \left(U_1 \mathcal{D}_1^{1/2} \right) \big|_{rows\ 1:(i-1)n_z}, \tag{5.13}$$

and compute the shifted latent state sequence:

$$\hat{X}_{i+1}^{(1)} = {\Gamma_{z_{i-1}}^{(1)}}^{\ \dagger} \left(Z_f^- / \begin{bmatrix} U_p^+ \\ Y_p^+ \\ U_f^- \end{bmatrix} \right). \tag{5.14}$$

Stage 2

Stage 2 proceed analogously to Stage 1, but focuses on the remaining intrinsic neural dynamics not explained by the behaviorally relevant states $x_i^{(1)}$. It operates on the residual future neural activity obtained after subtracting the contribution of the behaviorally relevant subspace, and extracts the dynamics that are irrelevant to behavior.

5. Isolate the component of future neural activity explained by $\hat{X}_i^{(1)}$ via an oblique projection that preserves input-driven activity:

$$\tilde{Y}_f = Y_f /_{\left[\begin{smallmatrix} U_p \\ U_f \end{smallmatrix}\right]} \hat{X}_i^{(1)} \, \Pi^{-1}_{\left[\begin{smallmatrix} U_p \\ U_f \end{smallmatrix}\right]}, \tag{5.15}$$

where $\Pi_{\begin{bmatrix} U_p \\ U_f \end{bmatrix}}$ is the orthogonal projection onto the row space of $\begin{bmatrix} U_p \\ U_f \end{bmatrix}$, defined in (3.3).

6. Subtract this component from Y_f to obtain the residual future neural activity:

$$Y_f' = Y_f - \tilde{Y}_f. \tag{5.16}$$

7. Project the residual future neural activity onto past neural activity and past input, along future input:

$$\hat{Y}_f' = Y_f' /_{U_f} \begin{bmatrix} U_p \\ Y_p \end{bmatrix}. \tag{5.17}$$

This oblique projection extracts the component of Y_f' that is predictable from past observations while excluding direct feedthrough of future input.

8. Compute the SVD of $\hat{Y}_f'$ and retain the top n_2 singular values:

$$\hat{Y}_f' = U' \mathcal{D}' V'^{\top} \approx U_2 \mathcal{D}_2 V_2^{\top}. \tag{5.18}$$

9. Compute the residual neural observability matrix $\Gamma_{y_i}^{(2)}$ as

$$\Gamma_{y_i}^{(2)} = U_2 \mathcal{D}_2^{1/2}, \tag{5.19}$$

and extract the remaining latent states at horizon i by applying ${\Gamma_{y_i}^{(2)}}^{\dagger}$ to the following orthogonal projection:

$$\hat{X}_i^{(2)} = {\Gamma_{y_i}^{(2)}}^{\dagger} \left(Y_f' / \begin{bmatrix} U_p \\ Y_p \\ U_f \end{bmatrix} \right). \tag{5.20}$$

10. Repeat at the shifted projection horizon $i+1$. Compute the reduced observability matrices by removing the last n_y rows:

$$\Gamma_{y_{i-1}}^{(2)} = {\Gamma_{y_i}^{(2)}}_{[1:(i-1)n_y,:]} = \left(U_2 \mathcal{D}_2^{1/2} \right) |_{rows\ 1:(i-1)n_y}. \tag{5.21}$$

Isolate the component of Y_f^- attributable to $\hat{X}_{i+1}^{(1)}$ via the same structure as in (5.15):

$$\tilde{Y}_f^- = Y_f^- /_{\begin{bmatrix} U_p^+ \\ U_f^- \end{bmatrix}} \hat{X}_{i+1}^{(1)} \, \Pi^{-1}_{\begin{bmatrix} U_p^+ \\ U_f^- \end{bmatrix}}, \tag{5.22}$$

and subtract it from Y_f^-:

$$Y_f'^- = Y_f^- - \tilde{Y}_f^-. \tag{5.23}$$

Project the residual onto past neural activity and past input, along future input:

$$\hat{Y}_f'^- = Y_f'^- /_{U_f^-} \begin{bmatrix} U_p^+ \\ Y_p^+ \end{bmatrix}, \tag{5.24}$$

and extract the shifted latent state sequence:

$$\hat{X}_{i+1}^{(2)} = {\Gamma_{y_{i-1}}^{(2)}}^{\dagger} \left(Y_f'^- / \begin{bmatrix} U_p^+ \\ Y_p^+ \\ U_f^- \end{bmatrix} \right). \tag{5.25}$$

Final Stage

Having extracted all latent state components at horizons i and $i+1$, the algorithm proceeds to identify the parameters of the state-space model in equation (5.1).

11. Concatenate the two state components into the full latent state sequences:

$$\hat{X}_i = \begin{bmatrix} \hat{X}_i^{(1)} \\ \hat{X}_i^{(2)} \end{bmatrix}, \quad \hat{X}_{i+1} = \begin{bmatrix} \hat{X}_{i+1}^{(1)} \\ \hat{X}_{i+1}^{(2)} \end{bmatrix}. \tag{5.26}$$

12. Estimate the behaviorally relevant dynamics matrix via least squares, regressing jointly onto the current state and input and retaining only the state columns:

$$A_{11} = \hat{X}_{i+1}^{(1)} \begin{bmatrix} \hat{X}_i^{(1)} \\ U_f \end{bmatrix}^{\dagger} \Big|_{columns\ 1:n_1}. \tag{5.27}$$

Estimate the remaining dynamics matrices analogously using the full latent state:

$$\begin{bmatrix} A_{21} & A_{22} \end{bmatrix} = \hat{X}_{i+1}^{(2)} \begin{bmatrix} \hat{X}_i \\ U_f \end{bmatrix}^{\dagger} \Big|_{columns\ 1:n_x} \tag{5.28}$$

and assemble the full dynamics matrix:

$$A = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}. \tag{5.29}$$

13. Estimate the observation matrices via least squares:

$$C_y = Y_i \begin{bmatrix} \hat{X}_i \\ U_f \end{bmatrix}^{\dagger} \Big|_{columns\ 1:n_x}, \qquad C_z = Z_i \begin{bmatrix} \hat{X}_i \\ U_f \end{bmatrix}^{\dagger} \Big|_{columns\ 1:n_x}. \tag{5.30}$$

14. Compute the residuals:

$$\begin{aligned} \hat{W}^{(1)} &= \hat{X}_{i+1}^{(1)} - \hat{X}_{i+1}^{(1)} / \begin{bmatrix} \hat{X}_i^{(1)} \\ U_f \end{bmatrix}, \\ \hat{W}^{(2)} &= \hat{X}_{i+1}^{(2)} - \hat{X}_{i+1}^{(2)} / \begin{bmatrix} \hat{X}_i \\ U_f \end{bmatrix}, \\ \hat{V} &= Y_i - Y_i / \begin{bmatrix} \hat{X}_i \\ U_f \end{bmatrix}. \end{aligned} \tag{5.31}$$

15. Estimate the process and observation noise covariances from the sample covariance of the stacked residuals:

$$\begin{bmatrix} Q & S \\ S^{\top} & R \end{bmatrix} = \frac{1}{j} \begin{bmatrix} \hat{W}^{(1)} \\ \hat{W}^{(2)} \\ \hat{V} \end{bmatrix} \begin{bmatrix} \hat{W}^{(1)} \\ \hat{W}^{(2)} \\ \hat{V} \end{bmatrix}^{\top}. \tag{5.32}$$

16. Estimate the input matrices B and D_y via a dedicated least squares procedure, similar to the one in [3, pp. 125–127], using the identified A, C_y, the extracted state sequences, and the observed neural activity.

17. Obtain the steady-state Kalman gain K (3.36) by solving the Riccati equation (3.35) as described in Section 3.4.2.

18. Propagate the Kalman filter over the training data to recursively recover latent states from neural observations and the known input, starting from $\hat{x}_0 = 0$:

$$\hat{x}_{k+1} = (A - KC_y)\hat{x}_k + (B - KD_y)u_k + Ky_k. \tag{5.33}$$

19. Finally, compute the direct feedthrough parameter D_z via an orthogonal projection:

$$D_z = \{z_k - C_z\hat{x}_k\}_{1:N} / \{u_k\}_{1:N}, \tag{5.34}$$

where $\{\alpha_k\}_{1:N}$ denotes the N-column matrix with column n containing α_n.

This concludes the learning phase of the algorithm, having identified all parameters A, B, C_y, D_y, C_z, D_z Q, R, and S of the input-driven state-space model. Note that the latent states sequence is estimated without looking at behavior z_k: the Kalman filter relies solely on neural activity and external inputs.

Simplified Cases

As in PSID, the algorithm accommodates edge cases by skipping stages whose corresponding state components are absent. When $n_1 = 0$, Stage 1 is omitted and Stage 2 identifies all latent dynamics from neural activity and input jointly, reducing to INDM [3]. When $n_2 = 0$, Stage 2 is omitted and only intrinsic behaviorally relevant dynamics are recovered in Stage 1. In both cases, the final stage operates on the state components that were extracted:

- If $n_1 = 0$ (no behaviorally relevant states): set $\hat{X}_i = \hat{X}_i^{(2)}$ and $\hat{X}_{i+1} = \hat{X}_{i+1}^{(2)}$.
- If $n_2 = 0$ (no behaviorally irrelevant states): set $\hat{X}_i = \hat{X}_i^{(1)}$ and $\hat{X}_{i+1} = \hat{X}_{i+1}^{(1)}$.

5.3 Algorithm Details

Having outlined the structure of the IPSID algorithm in Section 5.2, we now present the technical details. In the presence of external inputs u_k, the latent state dynamics and neural observations reflect both intrinsic activity and external input drive; the forward innovation formulation extends naturally to this input-driven setting and provides the theoretical foundation for

the oblique projections that characterize IPSID. We begin by deriving this extended formulation. Before presenting the full algorithm, we also illustrate the parameter identification step with a simple least-squares example, clarifying how the system matrices can be recovered once the latent states have been extracted. We then present the detailed algorithmic steps for state extraction and parameter identification. We then present the detailed algorithmic steps, referring to the PSID derivation in Section 4.3 and Chapter 4 of [3] for the latent state extraction in input-driven settings.

5.3.1 Forward Innovation Formulation

Similarly to PSID in Section 4.3.1, given the linear state-space formulation of equation (5.1) with known parameters, the Kalman filter yields optimal predictions in the minimum mean-square error sense (Section 3.4.1). Again, under the assumption of Gaussian noise, the best predictions of neural activity $\hat{y}_{k+1|k}$ and behavior $\hat{z}_{k+1|k}$ based on past observations $y_1, \ldots, y_k$ and inputs $u_1, \ldots, u_k$ are computed recursively as

$$\begin{cases} \hat{x}_{k+1|k} = A\hat{x}_{k|k-1} + Bu_k + K_k(y_k - C_y\hat{x}_{k|k-1} - D_y u_k) \\ \hat{y}_{k+1|k} = C_y\hat{x}_{k+1|k} + D_y u_{k+1} \\ \hat{z}_{k+1|k} = C_z\hat{x}_{k+1|k} + D_z u_{k+1} \end{cases}, \tag{5.35}$$

with K_k the time-varying Kalman gain. The innovation is now defined as

$$e_k \overset{\text{def}}{=} y_k - C_y\hat{x}_{k|k-1} - D_y u_k \tag{5.36}$$

and represents the component of current neural activity that cannot be predicted from either past observations or the current input.

Nevertheless, the model parameters are unknown and must be learned from data. Directly learning them from the formulation in (5.1) is unfeasible: there is again a circular dependence between the latent states x_k and the model parameters, since estimating the former requires the latter and vice versa, and the same problematic noise correlation structure described in Section 4.3.1. Assuming steady state has been reached, these issues are resolved by reformulating

(5.1) as the forward innovation formulation:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + Ke_k \\ y_k = C_y x_k + D_y u_k + e_k \\ z_k = C_z x_k + D_z u_k + \epsilon_k \end{cases}, \tag{5.37}$$

where $K = K_\infty$ is the steady-state Kalman gain defined in (3.30), e_k is the innovation process defined in (5.36), and $x_k \overset{\text{def}}{=} \hat{x}_{k|k-1}$.

Innovation e_k is white noise uncorrelated with past observations and inputs by construction, eliminating the noise correlation issue and ensuring that projections of future outputs onto past observations and inputs cleanly remove all noise terms. This reformulation also establishes two desirable properties that underlie the IPSID algorithm:

- The optimal estimate of the latent state x_k given past data can be expressed as a linear combination of past neural observations y_k and past inputs u_k.

- The optimal predictions of future neural activity y_k and behavior z_k are linear functions of the current state estimate x_k and current input u_k.

Together, these relationships allow the latent states to be extracted directly from data through subspace projections, without prior knowledge of the model parameters.

Explicit Formulation of State Linear Relationship

An explicit expression for the linear state relationship can be obtained by recursively substituting the state equation in (5.37): iterating backwards from time k, the predicted latent state x_k can be expressed as a linear combination of past neural observations, past inputs, and the initial state estimate. This result is formalized as Theorem 9 in [3, p. 101] for the combined

deterministic-stochastic case, which yields:

$$\hat{x}_k = \begin{bmatrix} A^k - \Omega_k \Gamma_{y_k} & \Delta^c_{u_k} - \Omega_k H_{y_k} & \Omega_k \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ u_0 \\ \vdots \\ u_{k-1} \\ y_0 \\ \vdots \\ y_{k-1} \end{bmatrix}, \tag{5.38}$$

where Γ_{y_k} is the neural observability matrix (4.12), $\Delta^c_{u_k}$ is the reversed deterministic reachability matrix (5.5), H_{y_k} is the deterministic Markov parameter matrix (5.6), and Ω_k is defined as

$$\Omega_k = \left(\Delta^c_{y_k} - A^k P_0 \Gamma^\top_{y_k}\right) \left(\Lambda_{y_k} - \Gamma_{y_k} P_0 \Gamma^\top_{y_k}\right)^{-1}. \tag{5.39}$$

This can be rewritten equivalently as

$$\hat{x}_k = \underbrace{\left(A^k - \Omega_k \Gamma_{y_k}\right)}_{n_x \times n_x} \hat{x}_0 + \underbrace{\left(\Delta^c_{u_k} - \Omega_k H_{y_k}\right)}_{n_x \times k n_u} \begin{bmatrix} u_0 \\ \vdots \\ u_{k-1} \end{bmatrix} + \underbrace{\Omega_k}_{n_x \times k n_y} \begin{bmatrix} y_0 \\ \vdots \\ y_{k-1} \end{bmatrix}, \tag{5.40}$$

which explicitly shows how the state estimate $\hat{x}_k$ decomposes into three contributions: propagation of the initial state, cumulative effects of past inputs, and information extracted from past neural observations.

For the special case where we initialize with zero initial conditions ($\hat{x}_0 = 0$ and $P_0 = 0$), the expression simplifies considerably. Under these assumptions, Ω_k reduces to

$$\Omega_k = \Delta^c_{y_k} \Lambda^{-1}_{y_k}, \tag{5.41}$$

and the state estimate becomes

$$\hat{x}_k = \left(\Delta^c_{u_k} - \Delta^c_{y_k} \Lambda^{-1}_{y_k} H_{y_k}\right) \begin{bmatrix} u_0 \\ \vdots \\ u_{k-1} \end{bmatrix} + \Delta^c_{y_k} \Lambda^{-1}_{y_k} \begin{bmatrix} y_0 \\ \vdots \\ y_{k-1} \end{bmatrix}. \tag{5.42}$$

In the pure stochastic case without inputs, this recovers the PSID result (4.44) from Theorem 6

in [3, p. 69]. The key insight is that $\hat{x}_k$ lies in the span of past inputs and neural observations, enabling its extraction through geometric projections onto these observed quantities, even before the model parameters are known.

5.3.2 Least Squares Parameter Estimate

The least-squares estimation procedure introduced in Section 4.3.2 extends directly to the IPSID framework. The key difference is that the regressor matrix Φ now includes the input sequence U_f alongside the extracted state sequence, reflecting the additional input term in the IPSID system equations (5.2). We detail this modification for A_{11}; the remaining system matrices follow analogously.

Problem Formulation

The behaviorally relevant latent state dynamics are now governed by:

$$x_k^{(1)} = A_{11}x_{k-1}^{(1)} + B_1 u_{k-1} + w_{k-1}^{(1)}. \tag{5.43}$$

Applying equation (5.43) at each time instant k in $\{i, \dots i+j-1\}$ and stacking column-wise yields the compact representation:

$$\hat{X}_{i+1}^{(1)} = A_{11}\hat{X}_i^{(1)} + B_1 U_f + W^{(1)}, \tag{5.44}$$

where U_f is the future input block Hankel matrix defined in Section 3.3 and $W^{(1)}$ collects the stacked process noise. This is a linear matrix regression with parameter matrix:

$$\Theta \stackrel{\text{def}}{=} \begin{bmatrix} A_{11} & B_1 \end{bmatrix} \in \mathbb{R}^{n_1 \times (n_1+n_u)}, \tag{5.45}$$

and regressor matrix:

$$\Phi \stackrel{\text{def}}{=} \begin{bmatrix} \hat{X}_i^{(1)} \\ U_f \end{bmatrix} \in \mathbb{R}^{(n_1+n_u)\times j}. \tag{5.46}$$

By construction, the first n_1 columns of Θ correspond to the state transition matrix, while the remaining n_u columns correspond to the input matrix B_1. Equation (5.44) takes the standard form:

$$\hat{X}_{i+1}^{(1)} = \Theta\Phi + W^{(1)}. \tag{5.47}$$

Provided $j \gg n_1 + n_u$ and Φ has full row rank, the problem is overdetermined and admits a least-squares solution.

Least Squares Solution

Minimizing the sum of squared residuals:

$$\hat{\Theta} = \arg\min_{\Theta} \left\| \hat{X}_{i+1}^{(1)} - \Theta\Phi \right\|_F^2, \tag{5.48}$$

the closed-form solution is:

$$\hat{\Theta} = \hat{X}_{i+1}^{(1)} \Phi^\top \left(\Phi\Phi^\top\right)^{-1} = \hat{X}_{i+1}^{(1)} \Phi^\dagger. \tag{5.49}$$

Substituting back the input-updated definitions of Θ and Φ, we get:

$$\begin{bmatrix} A_{11} & B_1 \end{bmatrix} = \hat{X}_{i+1}^{(1)} \begin{bmatrix} \hat{X}_i^{(1)} \\ U_f \end{bmatrix}^\dagger. \tag{5.50}$$

Since $\Theta \in \mathbb{R}^{n_1 \times (n_1 + n_u)}$, we extract the individual sub-block A_{11} by partitioning the columns:

$$A_{11} = \hat{X}_{i+1}^{(1)} \begin{bmatrix} \hat{X}_i^{(1)} \\ U_f \end{bmatrix}^\dagger \Bigg|_{columns\ 1:n_1} \tag{5.51}$$

This completes the estimation of A_{11}. The remaining system matrices are estimated analogously where applicable, with exceptions detailed in Section 5.3.4. With this framework in place, we now turn to the full algorithmic procedure.

5.3.3 Stage 1 and 2: Extracting Latent States

The latent state extraction procedure (Steps 1–10) follows the general subspace identification framework developed in [3], adapted here to the IPSID setting, and parallels the derivation presented for PSID in Sections 4.3.3 and 4.3.4. The key distinction is that IPSID replaces the orthogonal projections of PSID with oblique projections along the future input subspace, ensuring that the extracted states reflect intrinsic neural dynamics rather than input-driven responses. Specifically, the future behavior and neural activity are first projected obliquely to remove the contribution of measured inputs before the behaviorally relevant subspace is isolated via SVD,

mirroring the structure of the PSID derivation but operating on input-corrected quantities throughout. Here we make use of the linear relationship in (5.42), which extends the PSID result in (4.44) to the input-driven setting. We refer the reader to Chapter 4 of [3] for a detailed treatment of the underlying projections and matrix factorizations.

5.3.4 Final Stage: Parameter Identification

Steps 11–13. We concatenate the behaviorally relevant and behaviorally irrelevant latent states extracted in Stages 1 and 2 to form the full latent state sequences, as in (5.26). These sequences provide estimates of the true states at consecutive time steps. Therefore, the dynamics sub-blocks A_{11}, A_{21}, A_{22} defined in (5.2), as well as the observation matrices C_y and C_z, can therefore be recovered via least-squares regression using the extracted states and the observed data. The procedure follows Section 5.3.2, yielding the formulas in (5.27)–(5.30).

Step 14–15. The residuals in (5.31) provide estimates of the process and observation noise sequences by removing the deterministic state and input contributions from each equation. In each case, the orthogonal projection onto the corresponding regressors captures the component explained by the model, while the remaining part constitutes the noise estimate. Note that $\hat{W}^{(1)}$ only involves the behaviorally relevant states rather than the full latent state. This follows from the model structure, where $A_{12} = 0$ by construction, implying that $x_{k+1}^{(1)}$ depends only on $x_k^{(1)}$ and u_k, and not on $x_k^{(2)}$. The remaining two residuals instead use the full state, since they depend on the entire latent state.

Finally, the noise covariance matrices Q, S, and R are estimated from the sample covariance of the stacked residuals in (5.32).

Step 16. With A and C_y identified, the input matrices B and D_y can be estimated via least squares. Stacking the state and observation equations over the j training samples yields

$$\begin{bmatrix} \hat{X}_{i+1} \\ Y_i \end{bmatrix} = \begin{bmatrix} A \\ C_y \end{bmatrix} \hat{X}_i + \begin{bmatrix} B \\ D_y \end{bmatrix} U_f + \begin{bmatrix} \hat{W} \\ \hat{V} \end{bmatrix}. \tag{5.52}$$

Since the noise terms are uncorrelated with U_f, the matrices B and D_y can be obtained from the least-squares problem

$$B, D_y = \arg\min_{B,D_y} \left\| \begin{bmatrix} \hat{X}_{i+1} \\ Y_i \end{bmatrix} - \begin{bmatrix} A \\ C_y \end{bmatrix} \hat{X}_i - \begin{bmatrix} B \\ D_y \end{bmatrix} U_f \right\|_F^2. \tag{5.53}$$

The corresponding solution is given by the pseudoinverse

$$\begin{bmatrix} B \\ D_y \end{bmatrix} = \left(\begin{bmatrix} \hat{X}_{i+1} \\ Y_i \end{bmatrix} - \begin{bmatrix} A \\ C_y \end{bmatrix} \hat{X}_i \right) U_f^\dagger. \tag{5.54}$$

In practice, [3] also proposes a more robust formulation that constructs a structured regressor exploiting the block Toeplitz structure of the input-output map. This formulation is more accurate than the direct regression when the input Hankel matrix U_f is poorly conditioned, because the estimation uses information across all i prediction steps rather than relying on a single time slice. To this end, we define the matrix

$$\mathcal{K}(B, D_y) \overset{\text{def}}{=} \begin{bmatrix} \begin{bmatrix} B & \Gamma^\dagger_{y_{i-1}} H_{y_{i-1}} \end{bmatrix} - A\Gamma^\dagger_{y_i} H_{y_i} \\ \begin{bmatrix} D_y & 0 \end{bmatrix} - C_y \Gamma^\dagger_{y_i} H_{y_i} \end{bmatrix}, \tag{5.55}$$

where $\Gamma_{y_{i-1}}$, Γ_{y_i} are defined in (4.12), and $H_{y_{i-1}}$, H_{y_i} in (5.6). The matrix $\mathcal{K}(B, D_y)$ collects the dependence of the multi-step input-output relation on the unknown matrices B and D_y. The estimation problem can then be written as the structured least-squares regression

$$B, D_y = \arg\min_{B,\, D_y} \left\| \begin{bmatrix} \hat{\Gamma}^\dagger_{y_{i-1}} \hat{X}_{i+1} \\ Y_i \end{bmatrix} - \begin{bmatrix} A \\ C_y \end{bmatrix} \hat{\Gamma}^\dagger_{y_i} \hat{X}_i - \mathcal{K}(B, D_y) U_f \right\|_F^2. \tag{5.56}$$

Since $\mathcal{K}(B, D_y)$ depends linearly on B and D_y, the objective function is quadratic in the unknown parameters and therefore defines a convex least-squares problem. The minimum can be obtained either through a numerical optimization algorithm or, equivalently, by rewriting the expression inside the norm in (5.56) as a linear system in the entries of B and D_y and solving it via least squares. The explicit derivation of this linear system is given in [3, pp. 125–127].

Steps 17–19. At this point all parameters governing the state and neural observation equations have been identified. The steady-state Kalman gain K is obtained by solving the discrete

algebraic Riccati equation (3.35) with the identified noise covariances Q, R, S, as described in Section 3.4.2. With K in hand, the Kalman filter (5.33) is propagated over the N training samples to produce the filtered state estimates $\hat{x}_{k+1|k}$, initialized at $\hat{x}_{0|-1} = 0$. These are more accurate than the subspace-extracted states $\hat{X}_i$, since they are obtained by filtering through the full training record rather than a finite horizon of i observations. The parameter D_z is then recovered in (5.34) by projecting the residual behavior $z_k - C_z\hat{x}_{k+1|k}$ onto u_k: since $C_z\hat{x}_{k+1|k}$ accounts for the state-driven component of z_k, the residual isolates the direct input contribution $D_z u_k$, whose coefficient is then identified by least squares. This concludes the identification of all model parameters.

Remark 3 (Alternative estimation of C_z, D_z)**.** *Since neither C_z nor D_z enter the Kalman filter recursions* (5.33)*, both can be estimated in a final regression pass after the filter has been propagated over the training data, using the Kalman-filtered states $\hat{x}_{k+1|k}$ in place of the subspace-extracted states $\hat{X}_i$. Regressing z_k jointly onto $\hat{x}_{k+1|k}$ and u_k then yields both parameters simultaneously:*

$$\begin{bmatrix} C_z & D_z \end{bmatrix} = Z_k \begin{bmatrix} \hat{X}_{k+1|k} \\ U_k \end{bmatrix}^{\dagger}, \tag{5.57}$$

where Z_k and U_k are the matrices of stacked behavior observations and inputs respectively, defined in (3.23)*. This joint formulation is equivalent to the sequential procedure described above and implemented in the original IPSID paper [2], which estimates C_z first and then D_z from the residual $z_k - C_z\hat{x}_{k+1|k}$, but recovers both parameters in a single least squares problem.*

6

Numerical Simulations

Numerical simulations serve to validate the identification algorithms developed in the preceding chapters under controlled conditions in which the ground-truth model parameters are known exactly. This allows a direct quantitative assessment of how accurately PSID and IPSID recover the true latent dynamics from finite data, which is not possible with real neural recordings where the underlying model is inaccessible. In this chapter we present three sets of experiments: the first evaluates PSID on synthetic neural and behavioral data generated from random state-space models, assessing both the accuracy of eigenvalue identification and the convergence of identification error as a function of training sample size; the second extends the evaluation to IPSID, where an additional measured input u_k is present and the goal is to dissociate intrinsic behaviorally relevant dynamics from input-driven dynamics; the third directly compares PSID and IPSID to demonstrate the gains in identification accuracy achieved by IPSID when measured inputs are present.

All simulations were implemented in Python using the open-source `PyPSID` library [12], the reference implementation released alongside the PSID and IPSID papers [1, 2]. The library provides fitting routines that accept training time series $\{y_k : 0 \leq k < N\}$ and $\{z_k : 0 \leq k < N\}$ (and, for IPSID, additionally $\{u_k : 0 \leq k < N\}$), together with model structure parameters n_x, n_1, and i, and return an identified state-space model. All experiments below use these routines directly. Synthetic data are generated from random ground-truth models following the procedure of Supplementary Note 8 of [1] and its counterpart in [2], and detailed in Sections 6.1 and 6.2 respectively. Each identified model is then compared against the known

ground truth to quantify identification accuracy.

6.1 PSID

PSID is evaluated on synthetic data generated from random linear state-space models of the form (4.1), whose parameters are constructed explicitly and thus serve as ground truth for assessing identification accuracy.

6.1.1 Experimental Setup

Random Model Generation

The dimensions n_y, n_z, and n_x are drawn uniformly from $\{5, \ldots, 10\}$, $\{5, \ldots, 10\}$, and $\{1, \ldots, 10\}$, respectively, and n_1 is drawn uniformly from $\{1, \ldots, n_x\}$. Given these dimensions, the system matrices A, C_y, C_z, and the noise covariances Q, R, S are each generated according to the procedure described below.

The eigenvalues of A are sampled as follows. First, $\lfloor n_x/2 \rfloor$ points are drawn uniformly from the complex unit disk, and the complex conjugate of each is added to the set, yielding $2\lfloor n_x/2 \rfloor$ eigenvalues. If n_x is odd, one additional eigenvalue is drawn uniformly from the unit disk and its angle is set to whichever of 0 or π is closer. This construction guarantees that all eigenvalues are either real or appear in complex-conjugate pairs, satisfying the necessary condition for A to be a real-valued matrix. To designate n_1 eigenvalues as behaviorally relevant, the full set is partitioned into atomic units, each consisting of either a single real eigenvalue or a complex-conjugate pair, and these units are randomly permuted such that the first n_1 eigenvalues form a conjugate-closed set. The state transition matrix A is then constructed as

$$A = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}, \tag{6.1}$$

where $A_{11} \in \mathbb{R}^{n_1 \times n_1}$ is a block-diagonal matrix with the behaviorally relevant eigenvalues on the diagonal, and $A_{22} \in \mathbb{R}^{n_2 \times n_2}$, with $n_2 = n_x - n_1$, is constructed analogously from the remaining eigenvalues. Unlike the original procedure of [1], which takes $A_{21} = 0$, we draw A_{21} entry-wise from a standard normal distribution to allow coupling from the behaviorally relevant states to the irrelevant ones. Since A is block lower-triangular, its eigenvalues are those of A_{11} and A_{22} regardless of A_{21}, so the intended eigenvalue structure is preserved.

The observation matrix $C_y \in \mathbb{R}^{n_y \times n_x}$ is drawn entry-wise from a standard normal distribution with no structural constraints. The behavior matrix $C_z \in \mathbb{R}^{n_z \times n_x}$ has its first n_1 columns drawn entry-wise from a standard normal distribution and its remaining n_2 columns set to zero, reflecting the fact that only the behaviorally relevant component of the latent states $x_k^{(1)}$ contributes to behavior.

The noise covariances Q, R, and S are generated jointly. A random matrix Ω of size n_x+n_y is drawn entry-wise from a standard normal distribution and $L = \Omega\Omega^\top$ is formed, guaranteeing symmetric positive semi-definiteness. Two scaling exponents $a_1, a_2 \sim \mathrm{Uniform}(-1, 1)$ are drawn independently and the joint noise covariance is set as

$$\begin{bmatrix} Q & S \\ S^\top & R \end{bmatrix} = \begin{bmatrix} 10^{a_1} I_{n_x} \\ 10^{a_2} I_{n_y} \end{bmatrix} L \begin{bmatrix} 10^{a_1} I_{n_x} & 10^{a_2} I_{n_y} \end{bmatrix}. \tag{6.2}$$

This construction covers a wide range of noise regimes by independently varying the scale of the state noise w_k and the observation noise v_k across several orders of magnitude.

The behavior residual ϵ_k is generated by a separate autonomous state-space model, referred to as the behavior residual dynamics model, whose latent state dimension n_x' is drawn uniformly from $\{1, \ldots, 10\}$ and whose parameters are generated following the same procedure as the main model. This allows ϵ_k to be a general colored signal rather than white noise.

Once ϵ_k has been generated, the rows of C_z are rescaled to cover a diverse range of signal-to-noise ratios (SNR) for behavior. Each row corresponds to one dimension of z_k, i.e., one scalar component representing a measurable aspect of behavior. Specifically, since $z_k = C_z x_k + \epsilon_k$, the SNR for each behavior dimension d is defined as the ratio of the standard deviation of the shared component $C_z^{(d,:)} x$ to that of the residual $\epsilon^{(d)}$. A target SNR ρ is drawn uniformly on a log scale from $[1, 100]$, and the d-th row of C_z is rescaled so that

$$\frac{\mathrm{std}(C_z^{(d,:)} x)}{\mathrm{std}(\epsilon^{(d)})} = \rho, \tag{6.3}$$

covering scenarios ranging from behavior that is equally split between the shared and residual components ($\rho = 1$) to behavior strongly dominated by the shared latent state ($\rho = 100$).

Given the model parameters and the generated noise sequences, the neural activity and behavior are simulated by iterating through model (4.1), producing the time series $\{y_k : 0 \leq k < N\}$ and $\{z_k : 0 \leq k < N\}$.

Evaluation Metrics

Evaluating the accuracy of an identified state-space model against the known ground truth requires care due to the inherent non-uniqueness of state-space representations. A direct comparison of the identified model parameters against the true ones is complicated by the fact that a state-space model is only identifiable up to a similarity transform: any invertible change of basis in the latent state space yields an equivalent model. This means that even a perfectly identified model may have parameters that differ numerically from the true ones, rendering a direct comparison of matrices such as $\hat{A} - A$ or $\hat{C}_y - C_y$ meaningless without first aligning the two models to a common basis. The original paper [1] addresses this by finding the similarity transform that best aligns the Kalman-filtered states of the true and identified models before comparing all parameters; here, we restrict our evaluation to quantities that are invariant under similarity transforms and therefore do not require such alignment.

Eigenvalues are one such quantity, as they are preserved under any change of basis in the latent state space and provide a basis-independent measure of the identified dynamics. The primary evaluation metric is therefore the normalized eigenvalue error, which measures how accurately PSID recovers the behaviorally relevant dynamics. Let $\{\lambda_j\}_{j=1}^{n_1}$ and $\{\hat{\lambda}_j\}_{j=1}^{n_1}$ denote the eigenvalues of the true and identified blocks A_{11} and $\hat{A}_{11}$, respectively. An optimal one-to-one matching between true and identified eigenvalues is found by solving the linear assignment problem on the cost matrix $|\lambda_j - \hat{\lambda}_l|$, and the normalized error is defined as

$$\mathcal{E}_\lambda = \frac{\sum_j |\lambda_j - \hat{\lambda}_{\sigma(j)}|}{\sum_j |\lambda_j|}, \tag{6.4}$$

where σ is the optimal assignment. This metric is invariant to the arbitrary ordering of eigenvalues in the identified model and normalizes for the overall scale of the true dynamics.

The second metric is the coefficient of determination R^2 of behavior decoding, which measures how accurately behavior can be decoded from neural activity using the identified model. In the test phase, a Kalman filter associated with the learned model is applied to held-out neural activity y_k to extract the latent state estimates $\hat{x}_k$, from which behavior is decoded as $\hat{z}_k = C_z \hat{x}_k$. The decoding accuracy is quantified per behavior dimension as

$$R_d^2 = 1 - \frac{\sum_k (z_k^{(d)} - \hat{z}_k^{(d)})^2}{\sum_k (z_k^{(d)} - \bar{z}^{(d)})^2}, \quad d = 1, \ldots, n_z, \tag{6.5}$$

where $\bar{z}^{(d)}$ is the mean of the d-th dimension of the true behavior, and the overall R^2 is reported as the average across all n_z dimensions. An average $R^2 = 1$ indicates perfect decoding across all behavior dimensions, while $R^2 = 0$ indicates that the model explains no variance in behavior beyond the mean.

Experiments

For all experiments, the projection horizon is fixed at $i = 5$. This satisfies the two conditions $i \geq \lceil n_1/n_z \rceil$ and $i \geq \lceil n_x/n_y \rceil$ for the range of model dimensions considered in our simulations.

Experiment 1: Eigenvalue Recovery. To illustrate the ability of PSID to recover the behaviorally relevant dynamics from data, 20 random models are generated and fitted with $N = 50000$ samples each. The first half of the data is used for training and the second half is reserved for testing. The true and identified eigenvalues are displayed in the complex plane for a representative subset of 6 models, with line segments connecting each true behaviorally relevant eigenvalue to its closest identified counterpart, as determined by the optimal assignment described in Section 6.1.1. This visualization allows a direct assessment of how closely the identified dynamics match the true ones, both in terms of magnitude and phase. The normalized eigenvalue error $\mathcal{E}_\lambda$ is also reported for all 20 models.

Experiment 2: Behavior Decoding. To evaluate the practical utility of the identified model, behavior decoding accuracy is assessed across the same 20 random models used in Experiment 1, at a fixed training set size of $N = 50000$ samples. For each model, the first half of the data is used for training and the second half for testing. A Kalman filter associated with the learned model is applied to the test neural activity to extract latent state estimates, from which behavior is decoded as $\hat{z}_k = C_z \hat{x}_k$. The decoding accuracy is quantified using the coefficient of determination R^2, averaged across all n_z behavior dimensions, and reported for each model alongside the mean across models.

Experiment 3: Convergence of Eigenvalue Identification Error. To assess how identification accuracy depends on the amount of available training data, the normalized eigenvalue error $\mathcal{E}_\lambda$ is evaluated across the same 20 random models used in Experiments 1 and 2, for training set sizes $N \in \{500, 1000, 2000, 5000, 10000, 50000\}$. For each model, a realization of 50000 samples is generated; the first N samples are used for training and the second half of

the full realization serves as a fixed test set, ensuring that differences in $\mathcal{E}_\lambda$ across values of N reflect only the effect of training set size. The mean and standard error of $\mathcal{E}_\lambda$ across models are recorded for each N.

6.1.2 Results

Experiment 1: Eigenvalue Recovery. Figure 6.1 displays the true and identified eigenvalues in the complex plane for a representative subset of six models. In all six cases, the Stage 1 identified eigenvalues (green crosses) lie directly on top of the true behaviorally relevant eigenvalues (green circles), with connecting line segments indistinguishable at this scale, indicating accurate recovery of both the magnitude and phase of the behaviorally relevant dynamics. The Stage 2 identified eigenvalues (red crosses) also track the true behaviorally irrelevant eigenvalues (red circles) with comparable accuracy, although their recovery is not the primary objective of PSID.

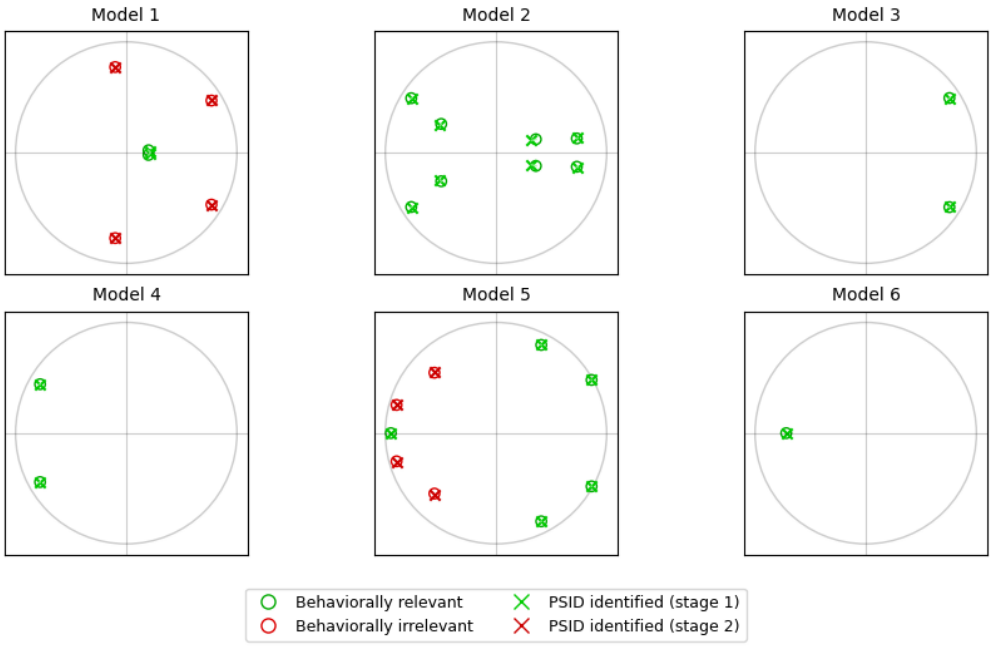


Figure 6.1: PSID eigenvalue recovery for a representative subset of 6 randomly generated models. Each panel shows the complex unit circle (gray), where the horizontal axis represents the real part and the vertical axis the imaginary part of each eigenvalue, with true behaviorally relevant eigenvalues (green circles) and true behaviorally irrelevant eigenvalues (red circles), alongside the eigenvalues identified by PSID Stage 1 (green crosses) and Stage 2 (red crosses). In all cases, identified eigenvalues closely match their true counterparts, confirming accurate recovery of both magnitude and phase across diverse dynamical regimes.

The normalized eigenvalue errors $\mathcal{E}_\lambda$ reported in Table 6.1 confirm this picture quantitatively. Across all 20 models, $\mathcal{E}_\lambda$ remains small, ranging from 0.0014 to 0.1277, with 17 out of 20 models achieving $\mathcal{E}_\lambda < 0.05$. The three best-recovered models (M3, M4, M6) achieve $\mathcal{E}_\lambda < 0.01$, and all three have small state dimensions with $n_1 = n_x \leq 2$, meaning the entire latent state is behaviorally relevant. In this setting, Stage 1 alone suffices to capture all system dynamics, reducing the identification problem to its simplest form. Overall, the results confirm that PSID reliably recovers the behaviorally relevant dynamics across a diverse set of random state-space models.

Experiment 2: Behavior Decoding. Figure 6.2 reports the behavior decoding R^2 across all 20 models, with the mean indicated by the dashed line. The corresponding values are listed in Table 6.1. Decoding accuracy varies considerably across models, ranging from 0.231 (M17) to 0.911 (M6), reflecting the diversity of the randomly generated state-space models. The mean R^2 of 0.66 indicates that, on average, the identified model explains a substantial fraction of behavior.

Decoding performance reflects eigenvalue recovery closely across models: M17, the weakest performer in Experiment 1, again achieves the lowest R^2, while M3, M4, and M6, which attained $\mathcal{E}_\lambda < 0.01$, reach $R^2 > 0.86$, with M6 attaining $R^2 = 0.911$. The inverse relationship between eigenvalue identification error and behavior decoding accuracy is strongly supported by a Pearson correlation of $r = -0.805$ ($p < 0.001$) across the 20 models, indicating that models which more accurately recover the behaviorally relevant dynamics consistently achieve higher decoding performance.

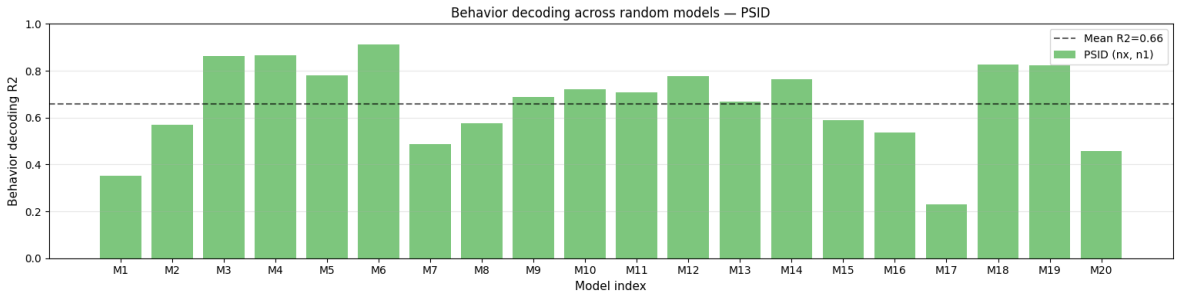


Figure 6.2: PSID behavior decoding R^2 across all 20 randomly generated models. The dashed line indicates the mean $R^2 = 0.66$, confirming that PSID accurately captures the behaviorally relevant dynamics on average. Performance varies substantially across models, with the best-performing models exceeding $R^2 = 0.8$ and the weakest falling below 0.4, reflecting the diversity of the randomly generated state-space models.

Experiment 3: Convergence of Eigenvalue Identification Error. Figure 6.3 reports the mean and standard error of $\mathcal{E}_\lambda$ across the 20 models as a function of training set size N. The error decreases monotonically with N, dropping from a mean of approximately 0.43 at $N = 500$ to near zero at $N = 50000$, confirming that PSID eigenvalue recovery improves consistently with the amount of available training data. The variance across models, reflected in the shaded band, also narrows substantially as N increases, indicating that performance becomes more reliable with larger datasets. At $N = 50000$, both the mean error and its standard error are close to zero, consistent with the near-perfect recovery observed in Experiments 1 and 2 at that sample size.

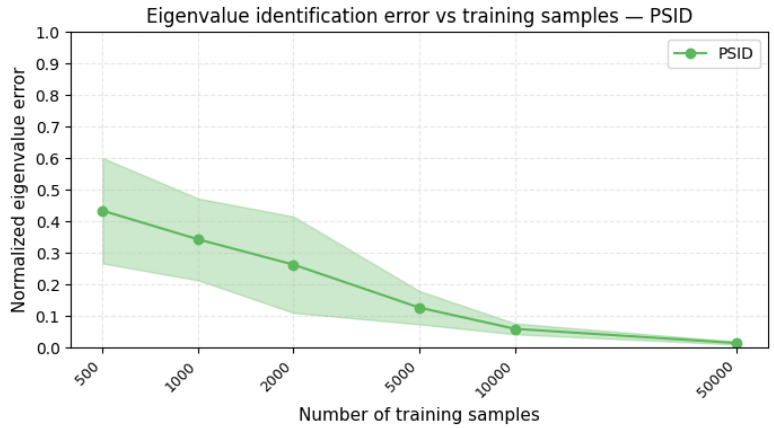


Figure 6.3: Convergence of PSID eigenvalue identification error with training set size. The normalized eigenvalue identification error $\mathcal{E}_\lambda$ is plotted as a function of training set size N, averaged across 20 randomly generated models. The shaded band indicates the standard error. The error decreases monotonically with N, reaching near zero at $N = 50000$, with the variance across models narrowing substantially at larger sample sizes, confirming that PSID eigenvalue recovery becomes both more accurate and more consistent as more training data is available.

These results establish PSID as a reliable method for recovering behaviorally relevant dynamics from neural population data, and provide a baseline against which the IPSID results are assessed in the following section.

6.2 IPSID

The following experiments mirror the structure of the PSID simulations, and evaluate IPSID on synthetic data generated from random linear state-space models of the form (5.1), driven by a measured input signal.

6.2.1 Experimental Setup

Random Model Generation

Random models are generated following the same procedure as in the PSID simulations (Section 6.1), with the following modifications to accommodate the input-driven setting.

The dimensions n_y, n_z, n_x, and n_1 are drawn as before. An input signal $u_k \in \mathbb{R}^{n_u}$ is additionally introduced, with n_u drawn uniformly from $\{1, \ldots, 4\}$. The state transition matrix A, observation matrix C_y, and behavior matrix C_z are constructed following the same procedure as in the PSID simulations, as are the noise covariances Q, R, and S. The input matrices $B \in \mathbb{R}^{n_x \times n_u}$, $D_y \in \mathbb{R}^{n_y \times n_u}$, and $D_z \in \mathbb{R}^{n_z \times n_u}$ are drawn entry-wise from a standard normal distribution. Unlike the original simulations in [2], where $D_z = 0$ in most experiments, here D_z is always taken to be non-zero, reflecting the more general input-driven setting. The behavior residual ϵ_k is generated as in Section 6.1. Once ϵ_k has been generated, the rows of C_z are rescaled following the same SNR procedure as in Section 6.1.

The input sequence $\{u_k : 0 \leq k < N\}$ is generated from a separate autonomous state-space model of the same form as the behavior residual dynamics model, with its own randomly drawn system matrices and noise covariances, and latent state dimension n_x'' drawn uniformly from $\{1, \ldots, 4\}$, ensuring that u_k is a general colored signal. Finally, the neural activity and behavior time series, respectively $\{y_k : 0 \leq k < N\}$ and $\{z_k : 0 \leq k < N\}$, are generated by iterating through model (5.1).

Note that since u_k is generated from a dynamical model with its own eigenvalues, the temporal structure of u_k is imprinted on the neural activity y_k both directly, through the feedthrough term $D_y u_k$, and indirectly, through the input-driven state evolution Bu_k. As a result, the eigenvalues of the input model appear in the dynamics of y_k and may be incorrectly identified as intrinsic behaviorally relevant dynamics by methods that do not account for input, such as PSID. This constitutes a challenging and realistic setting for evaluating IPSID's ability to dissociate intrinsic from input-driven dynamics.

Experiments

The same three experiments carried out for PSID are extended to the IPSID setting, with the input sequence provided to the algorithm at both training and test time. The projection horizon is fixed at $i = 5$ for all experiments, consistently with the PSID simulations, and the evaluation metrics are the same as defined in Section 6.1.1.

Experiment 1: Eigenvalue Recovery. The eigenvalue recovery experiment is repeated in the input-driven setting, to demonstrate the ability of IPSID to recover the behaviorally relevant dynamics in the presence of input. 20 random models are generated following the procedure described in Section 6.2.1 and fitted with $N = 50000$ samples each, with the first half used for training and the second half reserved for testing. The true and identified eigenvalues are displayed in the complex plane for a representative subset of 6 models. In addition to the behaviorally relevant and irrelevant eigenvalues, the eigenvalues of the input model are also shown, highlighting the potential for confounding between intrinsic and input-driven dynamics. The normalized eigenvalue error $\mathcal{E}_\lambda$ is reported for all 20 models.

Experiment 2: Behavior Decoding. Behavior decoding accuracy is assessed across the same 20 models used in Experiment 1, at a fixed training set size of $N = 50000$ samples, following a similar procedure as in the PSID case. Since the identified model includes a feedthrough term D_z, behavior is now decoded as $\hat{z}_k = C_z \hat{x}_k + D_z u_k$, where $\hat{x}_k$ are the Kalman-filtered state estimates.

Experiment 3: Convergence of Eigenvalue Identification Error. The convergence of $\mathcal{E}_\lambda$ with training set size is evaluated across the same 20 models as in Experiments 1 and 2, for $N \in \{500, 1000, 2000, 5000, 10000, 50000\}$, following an identical experimental procedure as in the PSID simulations. The mean and standard error of $\mathcal{E}_\lambda$ across models are reported for each value of N.

6.2.2 Results

Experiment 1: Eigenvalue Recovery. As in the PSID case, the Stage 1 identified eigenvalues (green crosses) lie directly on top of the true behaviorally relevant eigenvalues (green circles) across all six models displayed in Figure 6.4, with connecting line segments indistinguishable at this scale. The Stage 2 identified eigenvalues (red crosses) also show accurate tracking of the behaviorally irrelevant eigenvalues (red circles). Importantly, the input eigenvalues (blue circles) are not confused with the behaviorally relevant ones, confirming that IPSID correctly dissociates intrinsic from input-driven dynamics.

Table 6.1 provides quantitative confirmation of these results: $\mathcal{E}_\lambda$ remains small across all 20 models, with 18 out of 20 achieving $\mathcal{E}_\lambda < 0.05$. The three best-recovered models (M3, M8, M14) achieve $\mathcal{E}_\lambda < 0.001$, demonstrating that IPSID can recover the behaviorally relevant

dynamics with high precision in randomly generated input-driven state-space models. Additionally, the median error across all 20 models is 0.003, compared to 0.011 for PSID, indicating that IPSID generally achieves a more precise characterization of the behaviorally relevant dynamics.

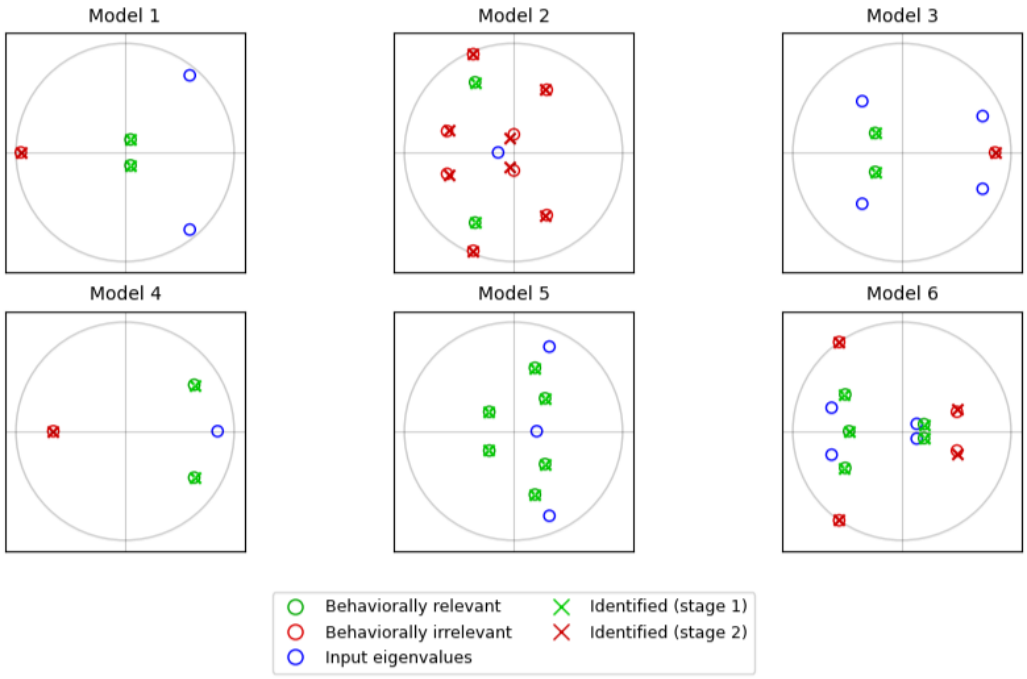


Figure 6.4: IPSID eigenvalue recovery for a representative subset of 6 randomly generated input-driven models. Each panel shows the complex unit circle (gray; real axis horizontal, imaginary axis vertical), with true behaviorally relevant eigenvalues (green circles), behaviorally irrelevant eigenvalues (red circles), and input eigenvalues (blue circles), alongside eigenvalues identified by IPSID Stage 1 (green crosses) and Stage 2 (red crosses). In all cases, identified eigenvalues closely match their true intrinsic counterparts, confirming accurate recovery. Furthermore, the behaviorally relevant eigenvalues are not confused with the input ones, demonstrating that IPSID correctly dissociates intrinsic from input-driven dynamics.

Experiment 2: Behavior Decoding. Behavior decoding R^2 across all 20 models is reported in Figure 6.5, with the corresponding values listed in Table 6.1. Decoding accuracy varies across models, ranging from 0.382 (M16) to 0.993 (M3), with a mean R^2 of 0.80, substantially higher than the 0.66 achieved by PSID. Notably, 8 out of 20 models achieve $R^2 > 0.90$, compared to only 1 in the PSID simulations, reflecting the benefit of explicitly accounting for input in the identification procedure.

As in PSID, decoding performance reflects eigenvalue recovery closely: M16 and M19, which exhibited the largest eigenvalue errors in Experiment 1, again rank among the weakest perform-

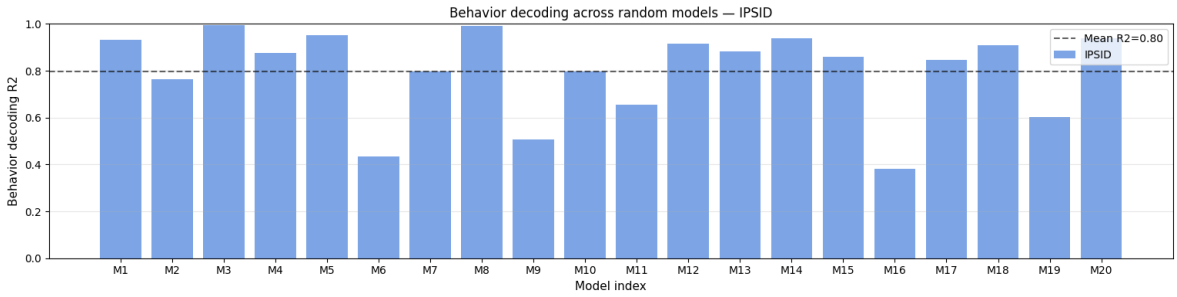


Figure 6.5: IPSID behavior decoding R^2 across all 20 randomly generated input-driven models. The dashed line marks the mean $R^2 = 0.80$, around which most models cluster, reflecting consistently strong decoding performance. Several models approach $R^2 = 1$, indicating near-perfect recovery of the behavioral signal, while only a handful fall below 0.5. This confirms that IPSID accurately captures the behaviorally relevant dynamics in input-driven settings.

ers, while M3 and M8, which achieved $\mathcal{E}_\lambda < 0.001$, reach $R^2 > 0.99$. The inverse relationship between eigenvalue identification error and behavior decoding accuracy is statistically significant across the 20 models (Pearson $r = -0.602$, $p < 0.01$; weaker than PSID due to the additional input contribution through $D_z u_k$ to behavioral prediction), confirming that accurate recovery of the behaviorally relevant dynamics is a reliable predictor of decoding performance.

Experiment 3: Convergence of Eigenvalue Identification Error. The mean and standard error of $\mathcal{E}_\lambda$ across the 20 models are shown in Figure 6.6 as a function of the

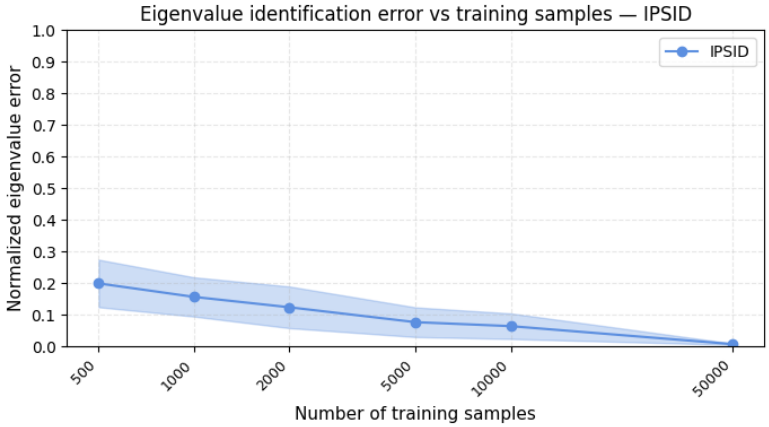


Figure 6.6: Convergence of IPSID eigenvalue identification error with training set size. The normalized eigenvalue identification error $\mathcal{E}_\lambda$ is plotted as a function of training set size N, averaged across 20 randomly generated input-driven models, with the shaded band indicating the standard error. The error decreases monotonically as N increases, approaching zero around $N = 50000$. Additionally, the uncertainty band narrows significantly with larger sample sizes, indicating that IPSID eigenvalue recovery becomes both more accurate and more consistent with larger training datasets.

training set size N. Similar to PSID, the error decreases monotonically with N, converging to near zero at $N = 50000$. However, IPSID starts from a substantially lower mean error of approximately 0.20 at $N = 500$, compared to 0.43 for PSID, and exhibits a narrower variance band throughout, indicating more consistent identification across models at all sample sizes. At $N = 50000$, the mean error approaches zero with minimal variability, consistent with the near-perfect recovery observed in Experiments 1 and 2.

Across all three experiments, IPSID demonstrates robust identification of the behaviorally relevant dynamics in the presence of measured input, with eigenvalue errors remaining small and decoding accuracy consistently high, showing in general stronger performance than PSID across a diverse set of randomly generated state-space models.

Table 6.1: Summary of PSID (left) and IPSID (right) simulation results across randomly generated state-space models. Each method was evaluated on an independent set of 20 models, characterized by their total latent state dimension n_x and behaviorally relevant state dimension n_1. For each model, the normalized eigenvalue identification error $\mathcal{E}_\lambda$ and the behavior decoding R^2 are reported.

PSID

Model	n_x	n_1	$\mathcal{E}_\lambda$	R^2
M1	6	2	0.0975	0.353
M2	8	8	0.0273	0.569
M3	2	2	0.0031	0.861
M4	2	2	0.0047	0.865
M5	9	5	0.0025	0.779
M6	1	1	0.0068	0.911
M7	6	4	0.0218	0.487
M8	7	7	0.0177	0.577
M9	3	3	0.0072	0.689
M10	10	4	0.0031	0.719
M11	3	1	0.0153	0.706
M12	8	4	0.0019	0.776
M13	10	10	0.0390	0.669
M14	9	8	0.0273	0.763
M15	7	3	0.0062	0.589
M16	8	8	0.0163	0.535
M17	9	4	0.1277	0.231
M18	2	2	0.0059	0.825
M19	3	3	0.0014	0.825
M20	4	4	0.0305	0.457

IPSID

Model	n_x	n_1	$\mathcal{E}_\lambda$	R^2
M1	3	2	0.0314	0.931
M2	10	2	0.0019	0.762
M3	3	2	0.0011	0.993
M4	3	2	0.0061	0.875
M5	6	6	0.0033	0.952
M6	9	5	0.0124	0.433
M7	4	4	0.0010	0.797
M8	9	7	0.0009	0.990
M9	10	2	0.0191	0.505
M10	3	2	0.0019	0.797
M11	3	3	0.0142	0.655
M12	5	4	0.0034	0.914
M13	10	2	0.0023	0.881
M14	9	1	0.0005	0.939
M15	4	2	0.0014	0.859
M16	9	4	0.5735	0.382
M17	5	5	0.0167	0.846
M18	2	2	0.0022	0.908
M19	8	8	0.1672	0.602
M20	10	4	0.0019	0.937

6.3 Direct Comparison of PSID and IPSID

The simulations presented so far evaluated PSID and IPSID on separate sets of randomly generated models, precluding a direct comparison. To further validate the advantage of explicitly accounting for input in the identification procedure, PSID is applied to the same input-driven datasets used in the IPSID simulations, simply ignoring the input sequence. This setting reflects a realistic scenario in which neural activity is driven by a measured external input that is not accounted for in the model. Under these conditions, PSID cannot dissociate intrinsic from input-driven dynamics, and its identified behaviorally relevant subspace is expected to be contaminated by input eigenvalues, leading to degraded eigenvalue recovery and decoding performance relative to IPSID.

Experimental Setup. The same 20 input-driven models and corresponding IPSID fits from Section 6.2 are used. PSID is additionally fitted on the same data with the input sequence withheld, so that both methods are evaluated under identical conditions except for the availability of the input. Training and test splits are identical to those used in the IPSID simulations: for each model, $N = 50000$ samples are available; the first half is used for training and the second half for testing. The metric $\mathcal{E}_\lambda$ is reported for the three representative models displayed in Figure 6.7.

Results. The true and identified eigenvalues for three representative models are displayed in Figure 6.7, with PSID and IPSID shown in the top and bottom rows respectively. In the PSID rows, the Stage 1 identified eigenvalues (green crosses) fail to consistently recover the true behaviorally relevant eigenvalues (green circles), with the identified eigenvalues visibly displaced toward the input eigenvalues (blue circles) in Models 1 and 3, directly illustrating the confounding of intrinsic dynamics by input-driven ones. In Model 2, a smaller but noticeable displacement is also present, and the Stage 2 identified eigenvalues (red crosses) likewise show signs of confounding with the input eigenvalues. In contrast, the IPSID rows show near-perfect recovery of both behaviorally relevant and irrelevant eigenvalues across all three models.

The normalized eigenvalue errors support this visual assessment: PSID achieves $\mathcal{E}_\lambda = 2.905$ and 1.962 on Models 1 and 3 respectively, compared to 0.031 and 0.001 for IPSID, while Model 2 shows a smaller difference between the two methods ($\mathcal{E}_\lambda = 0.011$ vs. 0.002), illustrating that the confounding effect is not uniform across all models.

These results confirm that explicitly accounting for input is essential for correctly dissociat-

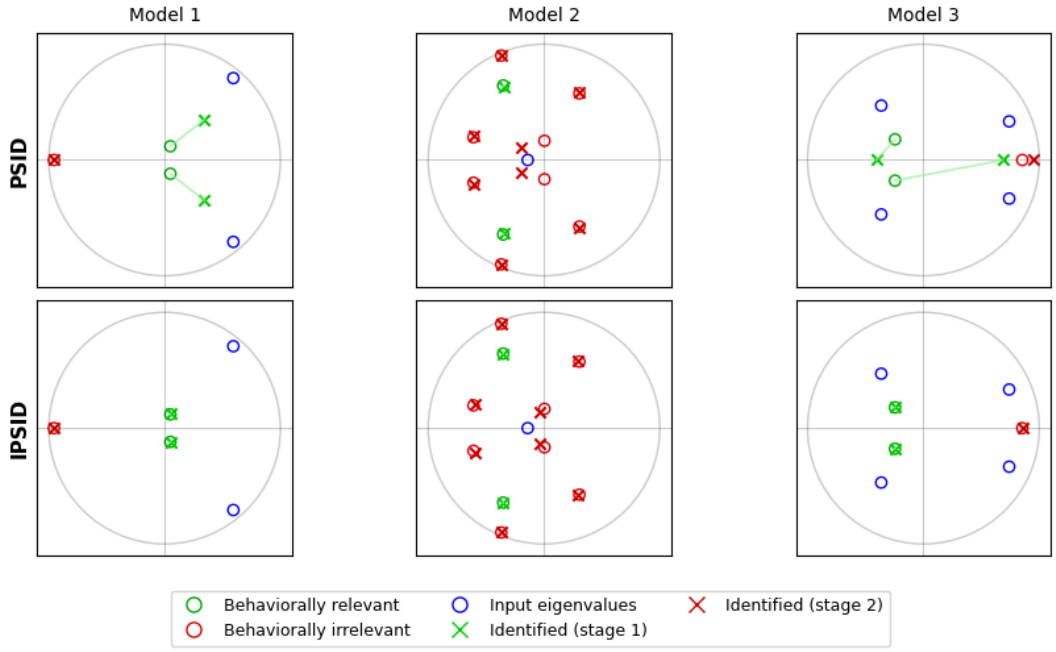


Figure 6.7: PSID versus IPSID eigenvalue recovery for a representative subset of 3 input-driven state-space models. PSID is applied with the input sequence withheld. True and identified eigenvalues are shown in the complex plane (gray; real axis horizontal, imaginary axis vertical), with PSID (top row) and IPSID (bottom row) displayed side by side. Green circles and crosses denote respectively the true and Stage 1 identified behaviorally relevant eigenvalues; red circles and crosses denote the true and Stage 2 identified behaviorally irrelevant eigenvalues; blue circles denote input eigenvalues. PSID eigenvalues are visibly displaced toward the input eigenvalues in Models 1 and 3, reflecting confounding of intrinsic and input-driven dynamics. On the contrary, IPSID recovers both sets of eigenvalues with near-perfect accuracy across all three models.

ing intrinsic behaviorally relevant dynamics from input-driven ones, and validate IPSID as the appropriate method when external inputs are present, concluding the numerical simulation analysis of this chapter.

7 Conclusions

This thesis examined two preferential subspace identification methods for linear neural population dynamics, Preferential Subspace Identification (PSID) and its input-driven extension, Input Preferential Subspace Identification (IPSID), with the dual goal of providing a rigorous mathematical treatment of the underlying theory and validating the resulting algorithms through controlled numerical experiments.

7.1 Summary of Contributions

The algorithmic steps of both PSID and IPSID were derived from the linear state-space framework of Van Overschee and De Moor [3], with each stage of the identification procedure rooted in the theory of oblique and orthogonal projections, block Hankel matrices, and stochastic realization. The two-stage structure was worked through in detail: Stage 1 extracts the behaviorally relevant component of the latent state by projecting the future neural output space onto the future behavior space along the past, and Stage 2 recovers the remaining latent dynamics from the residual. The projection geometry was examined in each case: orthogonal projections in PSID, and oblique projections in IPSID, where the presence of measured input requires projecting along the input subspace before isolating the behaviorally relevant dynamics. Explicit derivations of the parameter estimation steps were then given for each stage.

The numerical simulations of Chapter 6 validated both methods on synthetic data generated from random state-space models under conditions in which the ground-truth parameters

are known exactly. PSID reliably recovered the behaviorally relevant dynamics across a diverse set of randomly generated models, achieving a normalized eigenvalue error $\mathcal{E}_\lambda < 0.05$ for 17 out of 20 models at $N = 50000$ samples and a mean behavior decoding R^2 of 0.66, with $\mathcal{E}_\lambda$ decreasing monotonically with training set size. IPSID, evaluated on a separate set of input-driven models, matched or exceeded these results in all three experiments, achieving a mean R^2 of 0.80 and a median normalized eigenvalue error of 0.003, compared to 0.011 for PSID. The direct comparison of Section 6.3 further confirmed that ignoring measured inputs leads to systematic confounding of intrinsic and input-driven dynamics, with eigenvalue errors up to 100 times larger than those of IPSID. Together, these results establish PSID and IPSID as effective tools for behavior decoding and neural population modeling, with IPSID providing substantially more accurate identification when external inputs are present.

7.2 Limitations

The present work is subject to several limitations. The theoretical framework and simulations are both restricted to linear time-invariant state-space models with Gaussian noise. While this class may serve as a reasonable first-order approximation in certain regimes, it cannot capture nonlinear phenomena, non-stationarities, or non-Gaussian statistics that real neural population dynamics are known to exhibit. In particular, the stationarity assumption underlying the identification framework will not hold in the presence of learning, adaptation, or stimulation-induced plasticity. Second, the simulations assume that the state dimension n_x, the number of behaviorally relevant states n_1, and the horizon parameter i are known exactly, whereas in practice these must be estimated from data. Third, the evaluation relies entirely on synthetic data generated from the same class of models used for identification, a matched setting that is inherently favorable to the algorithms and does not assess their robustness when the true data-generating process deviates from the assumed model structure.

7.3 Future Directions

Several directions naturally extend the work presented in this thesis.

Validation on Real Neural Data. The most immediate step is to apply PSID and IPSID to real neural recordings from behaving animals, where the ground-truth model is inaccessible and the evaluation must rely on held-out decoding performance. The original papers [1, 2]

provide benchmarks on publicly available non-human primate datasets, including primary motor cortex recordings during reaching tasks [13] and dorsal premotor cortex recordings during sequential cursor control [14]. Reproducing and extending these results would constitute a direct empirical test of the identification framework beyond the synthetic setting.

Alternative SVD Weighting Schemes. A second direction concerns the SVD weighting scheme used in the subspace identification step: classical variants such as Numerical algorithms for Subspace State Space System Identification (N4SID), Multivariable Output-Error State sPace (MOESP), and Canonical Variate Analysis (CVA) [3, 15, 16, 17] apply different frequency weightings to the data, and their interaction with the preferential projection step in PSID and IPSID has not been systematically studied. Comparing these weighting schemes within the preferential identification setting could yield improvements in identification accuracy.

Nonlinear Extensions and Comparison with Neural Network Models. The preferential two-stage structure of PSID and IPSID is currently restricted to linear dynamics. RNN-based methods such as Latent Factor Analysis via Dynamical Systems (LFADS) [18] offer a nonlinear alternative within the non-preferential NDM framework, and the PSID paper [1] already demonstrated substantial computational and decoding advantages of PSID over LFADS. However, a nonlinear preferential method, one that embeds the two-stage prioritization into a nonlinear optimization framework, does not yet exist. The IPSID paper [2] identifies this as an open direction, and developing such a method would clarify the trade-offs between the analytical tractability of linear subspace identification and the representational capacity of nonlinear models.

Adaptation to Non-Stationary Dynamics. Extending PSID and IPSID to settings where the system matrices change over time, due to learning, neural plasticity, or closed-loop stimulation, would be a practically important direction. Adaptive or online variants of the identification procedure could track slow changes in the behaviorally relevant subspace, broadening the applicability of these methods to brain-machine interface settings. Initial steps in this direction have been taken for related neural dynamical models [19, 20], but extending such adaptive methods to the preferential identification setting remains open.

References

[1] O. G. Sani, H. Abbaspourazad, Y. T. Wong, B. Pesaran, and M. M. Shanechi, "Modeling behaviorally relevant neural dynamics enabled by preferential subspace identification," *Nature Neuroscience*, vol. 24, no. 1, pp. 140–149, 2021.

[2] P. Vahidi, O. G. Sani, and M. M. Shanechi, "Modeling and dissociation of intrinsic and input-driven neural population dynamics underlying behavior," *Proceedings of the National Academy of Sciences*, vol. 121, no. 7, p. e2212887121, 2024.

[3] P. Van Overschee and B. De Moor, *Subspace Identification for Linear Systems*. Springer, 1996.

[4] K. V. Shenoy, M. Sahani, and M. M. Churchland, "Cortical control of arm movements: a dynamical systems perspective," *Annual Review of Neuroscience*, vol. 36, pp. 337–359, 2013.

[5] W. Wu, J. E. Kulkarni, N. G. Hatsopoulos, and L. Paninski, "Neural decoding of hand motion using a linear state-space model with hidden states," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 17, no. 4, pp. 370–378, 2009.

[6] T. Katayama, *Subspace methods for system identification*. Springer, 2005.

[7] M. M. Shanechi, "Brain-machine interfaces from motor to mood," *Nature neuroscience*, vol. 22, no. 10, pp. 1554–1564, 2019.

[8] K. B. Hoang, I. R. Cassar, W. M. Grill, and D. A. Turner, "Biomarkers and stimulation algorithms for adaptive brain stimulation," *Frontiers in Neuroscience*, vol. 11, p. 564, 2017.

[9] G. H. Golub and C. F. Van Loan, *Matrix computations*, 4th ed. John Hopkins University Press, 2013.

[10] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[11] B. D. Anderson and J. B. Moore, *Optimal Filtering*. Prentice-Hall, 1979.

[12] O. G. Sani, P. Vahidi, and M. M. Shanechi. (2024) PyPSID: The Python library for (I)PSID. [Online]. Available: https://github.com/ShanechiLab/PyPSID

[13] J. E. O'Doherty, M. M. B. Cardoso, J. G. Makin, and P. N. Sabes. (2017) Nonhuman primate reaching with multichannel sensorimotor cortex electrophysiology. [Online]. Available: https://doi.org/10.5281/zenodo.583331

[14] M. G. Perich, P. N. Lawlor, K. P. Kording, and L. E. Miller. (2018) Extracellular neural recordings from macaque primary and dorsal premotor motor cortex during a sequential reaching task. [Online]. Available: https://doi.org/10.6080/K0FT8J72

[15] P. Van Overschee and B. De Moor, "N4SID: Subspace algorithms for the identification of combined deterministic-stochastic systems," *Automatica*, vol. 30, no. 1, pp. 75–93, 1994.

[16] M. Verhaegen, "Identification of the deterministic part of MIMO state space models," *Automatica*, vol. 30, no. 1, pp. 61–74, 1994.

[17] W. E. Larimore, "Canonical variate analysis in identification, filtering and adaptive control," in *Proceedings of the 29th IEEE Conference on Decision and Control*, Honolulu, HI, 1990.

[18] C. Pandarinath, D. J. O'Shea, J. Collins, R. Jozefowicz, S. D. Stavisky, J. C. Kao, E. M. Trautmann, M. T. Kaufman, S. I. Ryu, L. R. Hochberg, J. M. Henderson, K. V. Shenoy, L. F. Abbott, and D. Sussillo, "Inferring single-trial neural population dynamics using sequential auto-encoders," *Nature Methods*, vol. 15, no. 10, pp. 805–815, 2018.

[19] P. Ahmadipour, Y. Yang, E. F. Chang, and M. M. Shanechi, "Adaptive tracking of human ECoG network dynamics," *Journal of Neural Engineering*, vol. 18, p. 016011, 2020.

[20] Y. Yang, P. Ahmadipour, and M. M. Shanechi, "Adaptive latent state modeling of brain network dynamics with real-time learning rate optimization," *Journal of Neural Engineering*, vol. 18, p. 036013, 2020.

Acknowledgments

I would like to begin by thanking my supervisors Prof. Michele Allegra and Prof. Marco Formentin, for their availability, their guidance, and for giving me the freedom to make this project truly my own.

I am also deeply grateful to my parents, for their unwavering support through every success and every stumble across these university years (and long before them). And to my sister Leila, for the demanding, passionate, and unconditional love she somehow has for me.

To Chiara, for always making me feel understood, and for opening new doors in the world alongside me. To Eva, for making me feel at home. To Greta and Alberto, for being the best people I could have hoped to find in Padova, and for bringing so much joy into my life. To Dan, for surviving the math world with me, and to Manny, Libi, Manuel, and Conte, for all the fun memories of our BSC reunions.

To my volleyball teams in Padova, Udine and Eindhoven, for keeping my passion alive and always welcoming me back with open arms.

To my supervisor Yigit at ASML and all the people I met in the Netherlands, for six months of rain, new experiences, and the best excuse to travel Europe.

And to Marlon, for showing me the world and bringing me into his, one plane at a time, and for being always by my side.